

# SPI SLAVE UVM

**Presented by ahmed\_alaa**

**GITHUB**

# PART\_1 SPI

## CODE BUGS

### 1-WRONG ns

```
else begin
    if (received_address)
        ns = READ_ADD;
    else
        ns = READ_DATA;
end
```

### FIX

```
else begin
    if (received_address) ns = READ_DATA;
    else ns = READ_ADD;
end
```

### 2-IDLE OUTPUT HANDLING

```
case (cs)
    IDLE : begin
        rx_valid <= 0;
    end
```

### FIX

```
IDLE: begin
    rx_valid <= 0;
    MISO <= 0;
    rx_data <= 0; //FIX
end
```

# SPI MODULE

```
1 module SLAVE (  
2     MOSI,  
3     MISO,  
4     SS_n,  
5     clk,  
6     rst_n,  
7     rx_data,  
8     rx_valid,  
9     tx_data,  
10    tx_valid  
11 );  
12 input MOSI, clk, rst_n, SS_n, tx_valid;  
13 input [7:0] tx_data;  
14 output reg [9:0] rx_data;  
15 output reg rx_valid, MISO;  
16 localparam IDLE = 3'b000;  
17 localparam WRITE = 3'b001;  
18 localparam CHK_CMD = 3'b010;  
19 localparam READ_ADD = 3'b011;  
20 localparam READ_DATA = 3'b100;  
21  
22 reg [3:0] counter;  
23 reg      received_address;  
24  
25 reg [2:0] cs, ns;  
26 always @(posedge clk) begin  
27     if (~rst_n) begin  
28         cs <= IDLE;  
29     end else begin  
30         cs <= ns;  
31     end  
32 end
```

```
34 always @(*) begin  
35     case (cs)  
36         IDLE: begin  
37             if (SS_n) ns = IDLE;  
38             else ns = CHK_CMD;  
39         end  
40         CHK_CMD: begin  
41             if (SS_n) ns = IDLE;  
42             else begin  
43                 if (~MOSI) ns = WRITE;  
44                 else begin  
45                     if (received_address) ns = READ_DATA;  
46                     else ns = READ_ADD;  
47                 end  
48             end  
49         end  
50         WRITE: begin  
51             if (SS_n) ns = IDLE;  
52             else ns = WRITE;  
53         end  
54         READ_ADD: begin  
55             if (SS_n) ns = IDLE;  
56             else ns = READ_ADD;  
57         end  
58         READ_DATA: begin  
59             if (SS_n) ns = IDLE;  
60             else ns = READ_DATA;  
61         end  
62     endcase  
63 end
```

```
65 always @(posedge clk) begin  
66     if (~rst_n) begin  
67         rx_data <= 0;  
68         rx_valid <= 0;  
69         received_address <= 0;  
70         MISO <= 0;  
71     end else begin  
72         case (cs)  
73             IDLE: begin  
74                 rx_valid <= 0;  
75                 rx_data <= 0;  
76                 MISO <= 0; //BUG  
77             end  
78             CHK_CMD: begin  
79                 counter <= 10; //making counter =10 as if counter=0 then rx_valid is 1  
80             end  
81             WRITE: begin  
82                 if (counter > 0) begin //this means it needs to write  
83                     rx_data[counter-1] <= MOSI; //assign if write writing address or data  
84                     counter <= counter - 1;  
85                 end else rx_valid <= 1;  
86             end  
87             READ_ADD: begin  
88                 if (counter > 0) begin  
89                     rx_data[counter-1] <= MOSI;  
90                     counter <= counter - 1;  
91                 end else begin  
92                     rx_valid <= 1;  
93                     received_address <= 1;  
94                 end  
95             end  
96             READ_DATA: begin  
97                 if (tx_valid) begin  
98                     rx_valid <= 0;  
99                     if (counter > 0) begin  
100                        MISO <= tx_data[counter-1];  
101                        counter <= counter - 1;  
102                    end else begin  
103                        received_address <= 0;  
104                    end  
105                end else begin  
106                    if (counter > 0 && ~rx_valid) begin  
107                        rx_data[counter-1] <= MOSI;  
108                        counter <= counter - 1;  
109                    end else begin  
110                        rx_valid <= 1;  
111                        counter <= 8;  
112                    end  
113                end  
114            end  
115            default: begin  
116                rx_valid <= 0;  
117                rx_data <= 0;  
118                MISO <= 0;  
119            end  
120        endcase  
121    end  
122 end
```

# SPI\_IF\_TOP

```
1 interface spiif (  
2     clk  
3 );  
4     input clk;  
5     logic MOSI, MISO, SS_n, rst_n, tx_valid, rx_valid, MISO_exp, rx_valid_exp;  
6     logic [9:0] rx_data, rx_data_exp;  
7     logic [7:0] tx_data;  
8 endinterface  
9
```

```
36 initial begin  
37     uvm_config_db#(virtual spiif)::set(null, "uvm_test_top", "KEY", spiif_);  
38     run_test("spi_test");  
39 end
```

```
1 import uvm_pkg::*;  
2 `include "uvm_macros.svh"  
3 import spi_test_pkg::*;  
4 import config_pkg::*;  
5 module top ();  
6     bit clk;  
7     initial begin  
8         clk = 0;  
9         forever begin  
10             #10 clk = ~clk;  
11         end  
12     end  
13     spiif spiif_ (clk);  
14     SLAVE DUT (  
15         spiif_.MOSI,  
16         spiif_.MISO,  
17         spiif_.SS_n,  
18         spiif_.clk,  
19         spiif_.rst_n,  
20         spiif_.rx_data,  
21         spiif_.rx_valid,  
22         spiif_.tx_data,  
23         spiif_.tx_valid  
24     );  
25     SLAVE_GM _GM (  
26         spiif_.MOSI,  
27         spiif_.MISO_exp,  
28         spiif_.SS_n,  
29         spiif_.clk,  
30         spiif_.rst_n,  
31         spiif_.rx_data_exp,  
32         spiif_.rx_valid_exp,  
33         spiif_.tx_data,  
34         spiif_.tx_valid  
35     );  
36     initial begin
```

# SPI\_TEST\_CONFIG

```
1 package spi_test_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import spi_env_pkg::*;
5 import config_pkg::*;
6 import seq_rand_pkg::*;
7 import seq_rst_pkg::*;
8 class spi_test extends uvm_test;
9     `uvm_component_utils(spi_test)
10     spi_env spi_env_comp;
11     spi_config_obj spi_config_obj_test;
12     seq_rand randd;
13     seq_rst rst;
14     function new(string name = "spi_test", uvm_component parent = null);
15         super.new(name, parent);
16     endfunction
17     function void build_phase(uvm_phase phase);
18         super.build_phase(phase);
19         spi_env_comp = spi_env::type_id::create("spi_env_comp", this);
20         spi_config_obj_test = spi_config_obj::type_id::create("spi_config_comp", this);
21         randd = seq_rand::type_id::create("randd");
22         rst = seq_rst::type_id::create("rst");
23         if (~uvm_config_db#(virtual spiif)::get(this, "", "KEY", spi_config_obj_test.spi_config_vif))
24             `uvm_fatal("build_phase", "Test -Unable to get into virtual interface");
25         uvm_config_db#(spi_config_obj)::set(this, "*", "KEY", spi_config_obj_test);
26     endfunction
27     task run_phase(uvm_phase phase);
28         super.run_phase(phase);
29         phase.raise_objection(this);
30         rst.start(spi_env_comp.agent.sqr);
31         `uvm_info("run_phase", "randd_begin", UVM_LOW);
32         randd.start(spi_env_comp.agent.sqr);
33         `uvm_info("run_phase", "randd_end", UVM_LOW);
34         phase.drop_objection(this);
35     endtask
36 endclass
37 endpackage
```

```
1 package config_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 class spi_config_obj extends uvm_object;
5     `uvm_object_utils(spi_config_obj);
6     virtual spiif spi_config_vif;
7     function new(string name = "spi_config");
8         super.new(name);
9     endfunction //new()
10 endclass
11 endpackage
12
```



# SPI\_ENV\_AGENT

```
1 package spi_env_pkg;
2   import uvm_pkg::*;
3   import spi_driver_pkg::*;
4   import config_pkg::*;
5   import sequencer_pkg::*;
6   import spi_monitor_pkg::*;
7   import spi_seq_item_pkg::*;
8   import spi_agent_pkg::*;
9   import sb_pkg::*;
10  //import sb_pkg::*;
11  import cvg_pkg::*;
12  `include "uvm_macros.svh"
13  class spi_env extends uvm_env;
14    `uvm_component_utils(spi_env)
15    spi_agent agent;
16    spi_cover_gp cgp;
17    spi_seq_item seq_item;
18    spi_sb sb;
19    function new(string name = "spi_env", uvm_component parent = null);
20      super.new(name, parent);
21    endfunction //new()
22    function void build_phase(uvm_phase phase);
23      super.build_phase(phase);
24      agent = spi_agent::type_id::create("agent", this);
25      sb = spi_sb::type_id::create("sb", this);
26      cgp = spi_cover_gp::type_id::create("cgp", this);
27    endfunction
28    function void connect_phase(uvm_phase phase);
29      super.connect_phase(phase);
30      agent.agt_ap.connect(sb.sb_export);
31      agent.agt_ap.connect(cgp.cgp_export);
32    endfunction
33  endclass
34 endpackage
```

```
1 package spi_agent_pkg;
2   import uvm_pkg::*;
3   import spi_driver_pkg::*;
4   import config_pkg::*;
5   import sequencer_pkg::*;
6   import spi_monitor_pkg::*;
7   import spi_seq_item_pkg::*;
8   `include "uvm_macros.svh"
9   class spi_agent extends uvm_agent;
10     `uvm_component_utils(spi_agent)
11     spi_sequencer sqr;
12     spi_driver drv;
13     spi_config_obj cnfg;
14     spi_monitor mon;
15     uvm_analysis_port #(spi_seq_item) agt_ap;
16     function new(string name = "spi_agent", uvm_component parent = null);
17       super.new(name, parent);
18     endfunction
19     function void build_phase(uvm_phase phase);
20       super.build_phase(phase);
21       agt_ap = new("agt_ap", this);
22       if (~uvm_config_db#(spi_config_obj)::get(this, "", "KEY", cnfg)) begin
23         `uvm_fatal("build_phase", "unable to conf");
24       end
25       sqr = spi_sequencer::type_id::create("sqr", this);
26       drv = spi_driver::type_id::create("drv", this);
27       mon = spi_monitor::type_id::create("mon", this);
28     endfunction
29     function void connect_phase(uvm_phase phase);
30       super.connect_phase(phase);
31       drv.spi_driver_vif = cnfg.spi_config_vif;
32       mon.spi_monitor_if = cnfg.spi_config_vif;
33       drv.seq_item_port.connect(sqr.seq_item_export);
34       mon.mon_ap.connect(agt_ap);
35     endfunction
36   endclass
37 endpackage
```



# SPI\_DRIVER\_MONITOR

```
1 package spi_driver_pkg;
2 import uvm_pkg::*;
3 import config_pkg::*;
4 import spi_seq_item_pkg::*;
5 `include "uvm_macros.svh"
6 class spi_driver extends uvm_driver #(spi_seq_item);
7     `uvm_component_utils(spi_driver)
8     virtual spiif spi_driver_vif;
9     spi_config_obj spi_config_obj_driver;
10    spi_seq_item seq_item;
11    function new(string name = "spi_driver", uvm_component parent = null);
12        super.new(name, parent);
13    endfunction //new()
14    task run_phase(uvm_phase phase);
15        super.run_phase(phase);
16        forever begin
17            seq_item = spi_seq_item::type_id::create("seq_item_driver");
18            seq_item_port.get_next_item(seq_item);
19            spi_driver_vif.MOSI = seq_item.MOSI;
20            spi_driver_vif.rst_n = seq_item.rst_n;
21            spi_driver_vif.SS_n = seq_item.SS_n;
22            spi_driver_vif.tx_data = seq_item.tx_data;
23            spi_driver_vif.tx_valid = seq_item.tx_valid;
24            @(negedge spi_driver_vif.clk);
25            seq_item_port.item_done();
26        end
27    endtask
28 endclass
29 endpackage
30
```

```
1 package spi_monitor_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import spi_seq_item_pkg::*;
5 class spi_monitor extends uvm_monitor;
6     `uvm_component_utils(spi_monitor);
7     virtual spiif spi_monitor_if;
8     spi_seq_item seq_item;
9     uvm_analysis_port #(spi_seq_item) mon_ap;
10    function new(string name = "spi_monitor", uvm_component parent = null);
11        super.new(name, parent);
12    endfunction
13    function void build_phase(uvm_phase phase);
14        super.build_phase(phase);
15        mon_ap = new("mon_ap", this);
16    endfunction
17    task run_phase(uvm_phase phase);
18        super.run_phase(phase);
19        forever begin
20            seq_item = spi_seq_item::type_id::create("seq_item_spi");
21            //we must sample the outputs of the DUT
22            @(negedge spi_monitor_if.clk);
23            seq_item.MOSI = spi_monitor_if.MOSI;
24            seq_item.rst_n = spi_monitor_if.rst_n;
25            seq_item.MISO = spi_monitor_if.MISO;
26            seq_item.rx_data = spi_monitor_if.rx_data;
27            seq_item.rx_valid = spi_monitor_if.rx_valid;
28            seq_item.tx_data = spi_monitor_if.tx_data;
29            seq_item.tx_valid = spi_monitor_if.tx_valid;
30            seq_item.SS_n = spi_monitor_if.SS_n;
31            mon_ap.write(seq_item);
32            //also at the same time sample the outputs of the GOLDEN MODEL
33            seq_item.rx_data_exp = spi_monitor_if.rx_data_exp;
34            seq_item.rx_valid_exp = spi_monitor_if.rx_valid_exp;
35            seq_item.MISO_exp = spi_monitor_if.MISO_exp;
36            `uvm_info("run_phase", seq_item.convert2string(), UVM_HIGH);
37        end
38    endtask
39 endclass
40 endpackage
41
```

# SPI\_SEQ\_ITEM

```
1 package spi_seq_item_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 class spi_seq_item extends uvm_sequence_item;
5     `uvm_object_utils(spi_seq_item)
6     rand bit tx_valid;
7     rand bit rst_n;
8     rand bit [10:0] array_;
9     bit [10:0] prev_array;
10    bit [10:0] array_data;
11    bit SS_n;
12    bit MOSI;
13    bit is_read;
14    rand logic [7:0] tx_data;
15    logic MISO, rx_valid, rx_valid_exp, MISO_exp;
16    logic [9:0] rx_data, rx_data_exp;
17    int counter;
18    int index;
19    bit communication;
20    function new(string name = "spi_seq_item");
21        super.new(name);
22        counter = 0;
23        index = 10;
24        communication = 0;
25        array_data = array_;
26    endfunction
```

```
27    constraint array__ {
28        {array_[10], array_[9], array_[8]} inside {3'b000, 3'b001, 3'b110, 3'b111};
29        array_ != prev_array;
30    }
31    constraint reset {
32        rst_n dist {
33            1 := 98,
34            0 := 2
35        };
36    }
```



# SPI\_SEQ\_ITEM

```
1 function void pre_randomize();
2   if (SS_n) begin
3     array_data = array_;
4     prev_array = array_;
5   end
6   is_read = ({array_data[10], array_data[9], array_data[8]} == 3'b111); //indicates that it is reading
7   SS_n = ~communication; //every time i change my array if there is a transition in SS_n
8   if (is_read) begin
9     if (counter == 23) begin
10      communication = 0;
11      counter = 0;
12    end else begin
13      communication = 1;
14      counter++;
15    end
16  end else begin
17    if (counter == 13) begin
18      communication = 0;
19      counter = 0;
20    end else begin
21      communication = 1;
22      counter++;
23    end
24  end
25  if (is_read) begin
26    tx_valid = 1;
27  end else tx_valid = 0;
28  if (~SS_n) begin
29    if (index >= 0) begin
30      MOSI = array_data[index];
31      index = index - 1;
32    end else begin
33      MOSI = 0;
34      index = 10;
35    end
36  end else begin
37    MOSI = 0;
38    index = 10;
39  end
40 endfunction
```

```
77 function string convert2string();
78   return $sformatf(
79     "%s MOSI=%0d, MISO=%0b, SS_n=0b%0b, rst_n=0b%0b, rx_data=0b%0b, rx_valid=0b%0b, tx_data=0b%0b, tx_valid=0b%0b",
80     super.convert2string(),
81     MOSI,
82     MISO,
83     SS_n,
84     rst_n,
85     rx_data,
86     rx_valid,
87     tx_data,
88     tx_valid
89   );
90 endfunction
91 function string convert2string_stimulus();
92   return $sformatf(
93     "MOSI=%0d, MISO=%0b, SS_n=0b%0b, rst_n=0b%0b, rx_data=0b%0b, rx_valid=0b%0b, tx_data=0b%0b, tx_valid=0b%0b",
94     MOSI,
95     MISO,
96     SS_n,
97     rst_n,
98     rx_data,
99     rx_valid,
100    tx_data,
101    tx_valid
102  );
103 endfunction
104 endclass
105 endpackage
```

# SPI\_rand|rst\_seq\_Sqr

```
1 package seq_rand_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import spi_seq_item_pkg::*;
5 class seq_rand extends uvm_sequence #(spi_seq_item);
6     `uvm_object_utils(seq_rand);
7     spi_seq_item seq_item;
8     function new(string name = "seq_rand");
9         super.new(name);
10    endfunction
11    task body;
12        seq_item = spi_seq_item::type_id::create("seq_item_rand");
13        repeat (10000) begin
14            start_item(seq_item);
15            assert (seq_item.randomize());
16            finish_item(seq_item);
17        end
18    endtask
19 endclass //seq_rand
20 endpackage
```

```
1 package seq_rst_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import spi_seq_item_pkg::*;
5 class seq_rst extends uvm_sequence #(spi_seq_item);
6     `uvm_object_utils(seq_rst);
7     spi_seq_item seq_item;
8     function new(string name = "seq_rst");
9         super.new(name);
10    endfunction
11    task body;
12        seq_item = spi_seq_item::type_id::create("seq_item_rst");
13        repeat (10) begin
14            start_item(seq_item);
15            seq_item.rst_n = 0;
16            finish_item(seq_item);
17        end
18    endtask
19 endclass //seq_rst
20 endpackage
```

```
1 package sequencer_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import spi_seq_item_pkg::*;
5 import seq_rand_pkg::*;
6 class spi_sequencer extends uvm_sequencer #(spi_seq_item);
7     `uvm_component_utils(spi_sequencer)
8     function new(string name = "spi_sequencer", uvm_component parent = null);
9         super.new(name, parent);
10    endfunction
11 endclass
12 endpackage
```

# SPI\_GOLDEN\_MODEL

```
1  module SLAVE_GM (
2      MOSI,
3      MISO,
4      SS_n,
5      clk,
6      rst_n,
7      rx_data,
8      rx_valid,
9      tx_data,
10     tx_valid
11 );
12     localparam IDLE = 3'b000;
13     localparam WRITE = 3'b001;
14     localparam CHK_CMD = 3'b010;
15     localparam READ_ADD = 3'b011;
16     localparam READ_DATA = 3'b100;
17     input MOSI, clk, rst_n, SS_n, tx_valid;
18     input [7:0] tx_data;
19     output reg [9:0] rx_data;
20     output reg rx_valid, MISO;
21     reg [3:0] counter;
22     reg        ADRESS_RECIVED; //to diff between read adress and read data
23     reg [2:0] cs, ns;
24     // store
25     always @(posedge clk) begin
26         if (~rst_n) begin
27             cs <= IDLE;
28         end else begin
29             cs <= ns;
30         end
31     end
32     // ns logic
33     always @(*) begin
34         case (cs)
35             IDLE: begin
36                 if (SS_n) begin
37                     ns = IDLE;
38                 end else begin
39                     ns = CHK_CMD;
40                 end
41             end
42             CHK_CMD: begin
43                 if (SS_n) begin
44                     ns = IDLE;
45                 end else if (~SS_n && ~MOSI) begin
46                     ns = WRITE;
47                 end else if (~SS_n && MOSI && ~ADRESS_RECIVED) begin
48                     ns = READ_ADD;
49                 end else ns = READ_DATA;
50             end
51             WRITE: begin
52                 if (SS_n) begin
53                     ns = IDLE;
54                 end else ns = WRITE;
55             end
56             READ_ADD: begin
57                 if (SS_n) begin
58                     ns = IDLE;
59                 end else ns = READ_ADD;
60             end
61             READ_DATA: begin
62                 if (SS_n) begin
63                     ns = IDLE;
64                 end else ns = READ_DATA;
65             end
66         endcase
67     end
68     //output logic
```

# SPI\_GOLDEN\_MODEL

```
69 always @(posedge clk) begin
70     if (~rst_n) begin
71         rx_data <= 0;
72         rx_valid <= 0;
73         ADRESS_RECIVED <= 0;
74         MISO <= 0;
75     end else begin
76         case (cs)
77             IDLE: begin
78                 rx_valid <= 0;
79                 MISO <= 0; //missing
80                 rx_data <= 0; //bug
81             end
82             CHK_CMD: begin
83                 counter <= 10; //making counter =10 as if counter=0 then rx_valid is 1
84             end
85             WRITE: begin
86                 if (counter > 0) begin //this means it needs to write
87                     rx_data[counter-1] <= MOSI; //assign if write writing address or data
88                     counter <= counter - 1;
89                 end else rx_valid <= 1;
90             end
91             READ_ADD: begin
92                 if (counter > 0) begin
93                     rx_data[counter-1] <= MOSI;
94                     counter <= counter - 1;
95                 end else begin
96                     rx_valid <= 1;
97                     ADRESS_RECIVED <= 1;
98                 end
99             end
100         endcase
101     end
102 end
103
104 READ_DATA: begin
105     if (tx_valid) begin
106         rx_valid <= 0;
107         if (counter > 0) begin
108             MISO <= tx_data[counter-1];
109             counter <= counter - 1;
110         end else begin
111             ADRESS_RECIVED <= 0;
112         end
113     end else begin
114         if (counter > 0 && ~rx_valid) begin
115             rx_data[counter-1] <= MOSI;
116             counter <= counter - 1;
117         end else begin
118             rx_valid <= 1;
119             counter <= 8;
120         end
121     end
122 end
123 endmodule
```



# SPI\_SB

```
1 package sb_pkg;
2   import config_pkg::*;
3   import spi_seq_item_pkg::*;
4   import uvm_pkg::*;
5   import spi_agent_pkg::*;
6   import seq_rand_pkg::*;
7   `include "uvm_macros.svh"
8   class spi_sb extends uvm_scoreboard;
9     `uvm_component_utils(spi_sb);
10    uvm_analysis_export #(spi_seq_item) sb_export;
11    uvm_tlm_analysis_fifo #(spi_seq_item) sb_fifo;
12    spi_seq_item seq_item;
13    int error_count = 0;
14    int correct_count = 0;
15    function new(string name = "scoreboard", uvm_component parent = null);
16      super.new(name, parent);
17    endfunction
18    function void build_phase(uvm_phase phase);
19      super.build_phase(phase);
20      sb_export = new("sb_export", this);
21      sb_fifo = new("sb_fifo", this);
22    endfunction
23    function void connect_phase(uvm_phase phase);
24      super.connect_phase(phase);
25      sb_export.connect(sb_fifo.analysis_export);
26    endfunction
27    task run_phase(uvm_phase phase);
28      super.run_phase(phase);
29      forever begin
30        sb_fifo.get(seq_item);
31        if (seq_item.rx_data != seq_item.rx_data_exp || seq_item.rx_valid != seq_item.rx_valid_exp || seq_item.MISO != seq_item.MISO_exp)
32          `uvm_error("run_phase", $sformatf(
33            "comparason failed,DUT_transaction=%s refrence data=0b%0b refrence rx_valid=0b%0b refrence MISO=0b%0b",
34            seq_item.convert2string(),
35            seq_item.rx_data_exp,
36            seq_item.rx_valid_exp,
37            seq_item.MISO_exp
38          ));
39        error_count = error_count + 1;
40      end else begin
41        correct_count = correct_count + 1;
42      end
43    end
44  endtask
45  function void report_phase(uvm_phase phase);
46    super.report_phase(phase);
47    `uvm_info("report_phase", $sformatf("Total correct:%d", correct_count), UVM_MEDIUM);
48    `uvm_info("report_phase", $sformatf("Total false:%d", error_count), UVM_MEDIUM);
49  endfunction
50 endclass
51 endpackage
52
```



# SPI\_ASSERTIONS

```
119 property rst_n_prop;  
120   @(posedge clk) !rst_n | => {  
121     ~MISO && ~rx_valid && ~rx_data  
122   };  
123 endproperty  
124 property VALID_wr_add_seq;  
125   @(posedge clk) disable iff (!rst_n) ($fell(  
126     SS_n  
127   ) ##1(!MOSI)) [* 3]) |-> ##10 (rx_valid && SS_n);  
128 endproperty  
129 property VALID_wr_data_seq;  
130   @(posedge clk) disable iff (!rst_n) ($fell(SS_n) ##1 (!MOSI) ##1 (!MOSI) ##1 (MOSI) ) | => ##10 (rx_valid && $rose(SS_n)[->1])  
131 endproperty  
132 property VALID_rd_add_seq;  
133   @(posedge clk) disable iff (!rst_n)  
134   ($fell(SS_n) ##1 (MOSI) ##1 (MOSI) ##1 (!MOSI) ) | => ##10 (rx_valid && $rose(SS_n)[->1]);  
135 endproperty  
136   property VALID_rd_data_seq;  
137     @(posedge clk) disable iff (!rst_n)($fell(SS_n) ##1 (MOSI) ##1 (MOSI) ##1 (MOSI) ) | => ##10 (rx_valid && $rose(SS_n)[->1]);  
138   endproperty  
139   property first;  
140   disable iff (!rst_n)  
141   @(posedge clk) (cs == IDLE && !SS_n) |-> (ns == CHK_CMD);  
142 endproperty  
143   property sec;  
144   disable iff (!rst_n)  
145   @(posedge clk) cs == CHK_CMD |-> ns == WRITE || READ_ADD || READ_DATA;  
146 endproperty  
147   property third;  
148   disable iff (!rst_n)  
149   @(posedge clk) (cs == WRITE && SS_n) |-> ns == IDLE;  
150 endproperty
```

151

152 ✓

153

154

155

156 ✓

157

158

```
property forth;
```

```
  disable iff (!rst_n)
```

```
    @(posedge clk) cs == READ_ADD && SS_n |-> ns == IDLE;
```

```
  endproperty
```

```
  property fifth;
```

```
    disable iff (!rst_n)
```

```
      @(posedge clk) cs == READ_DATA && SS_n |-> ns == IDLE;
```

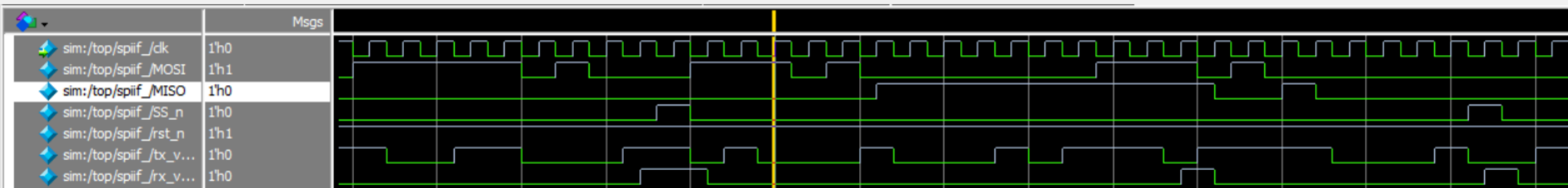
```
  endproperty
```

# SPI\_ASSERTIONS

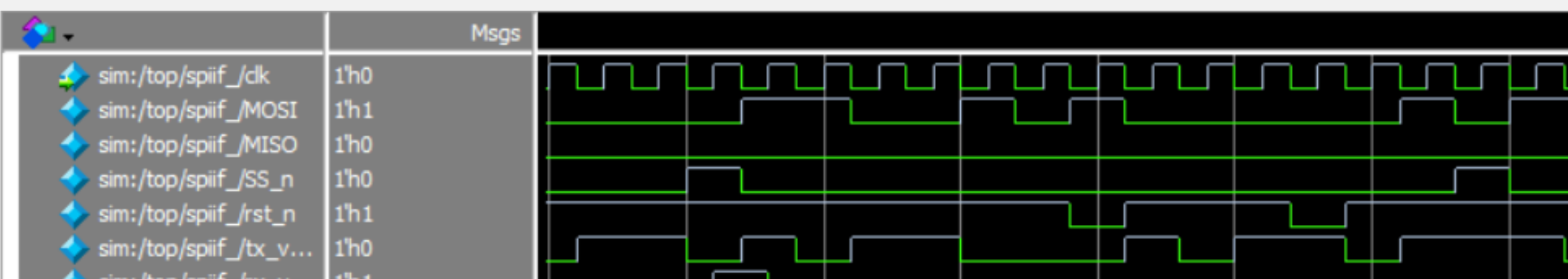
| ASSERTION_NAME                     | DESCRIPTION                                                               | ASSERTION                                                   |                   |                                        |                                                                                            |
|------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------|-------------------|----------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Reset Assertions</b>            |                                                                           |                                                             |                   |                                        |                                                                                            |
| rst_n_prop                         | When reset is asserted, all outputs should be cleared on next clock cycle | @(posedge clk) !rst_n   => {~MISO && ~rx_valid && ~rx_data} | VALID_wr_add_seq  | Valid write address sequence detection | \$fell(SS_n) ##1(!MOSI) [* 3])  -> ##10 (rx_valid && SS_n)                                 |
| <b>State Transition Assertions</b> |                                                                           |                                                             |                   |                                        |                                                                                            |
| first                              | In IDLE state with SS_n low, next state should be CHK_CMD                 | (cs == IDLE && !SS_n)  -> (ns == CHK_CMD)                   | VALID_wr_data_seq | Valid write data sequence detection    | \$fell(SS_n) ##1(\$past(!MOSI)) [* 2] ##1 (\$past(MOSI)))  -> ##10 (rx_valid && SS_n)      |
| sec                                | In CHK_CMD state, next state should be WRITE, READ_ADD, or READ_DATA      | cs == CHK_CMD  -> ns == WRITE    READ_ADD    READ_DATA      |                   |                                        |                                                                                            |
| third                              | In WRITE state with SS_n high, next state should be IDLE                  | (cs == WRITE && SS_n)  -> ns == IDLE                        | VALID_rd_add_seq  | Valid read address sequence detection  | \$fell(SS_n) ##1 (MOSI) ##1 (MOSI) ##1 (!MOSI) )   => ##10 (rx_valid && \$rose(SS_n)[->1]) |
| forth                              | In READ_ADD state with SS_n high, next state should be IDLE               | cs == READ_ADD && SS_n  -> ns == IDLE                       |                   |                                        |                                                                                            |
| fifth                              | In READ_DATA state with SS_n high, next state should be IDLE              | cs == READ_DATA && SS_n  -> ns == IDLE                      | VALID_rd_data_seq | Valid read data sequence detection     | \$fell(SS_n) ##1 (MOSI) ##1 (MOSI) ##1 (MOSI) )   => ##10 (rx_valid && \$rose(SS_n)[->1])  |

# SPI\_WAVEFORM

## RD\_DATA

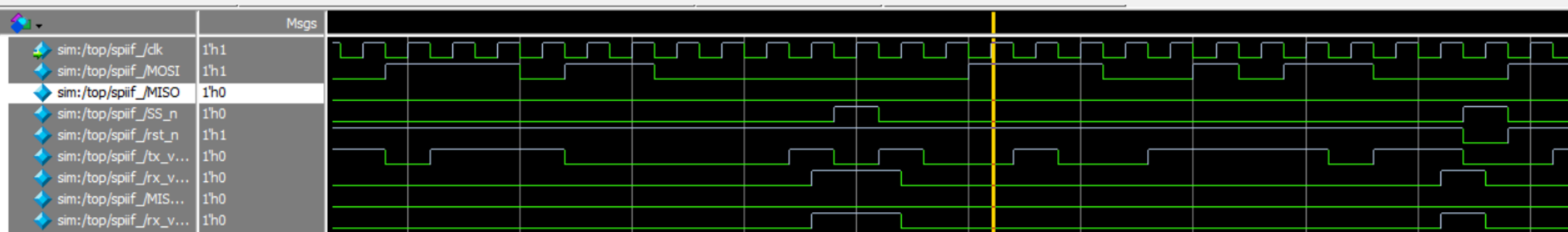


## RD\_ADD

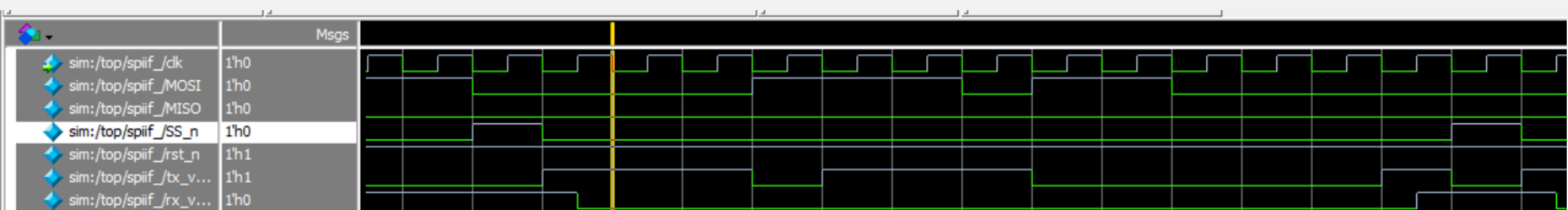


# SPI\_WAVEFORM

## WR\_DATA



## WR\_ADD



# SPI\_FUNCTIONAL COVERAGE

|                                     |     |           |         |         |   |         |   |
|-------------------------------------|-----|-----------|---------|---------|---|---------|---|
| /cvg_pkg/spi_cover_gp               |     |           |         | 100.00% |   |         |   |
| TYPE cvr_gp                         | 100 | 100.00... | 100.00% |         | ✓ | auto(0) |   |
| CVP cvr_gp::rx_data_all             | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| CVP cvr_gp::SS_n_seq                | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| CVP cvr_gp::MOSI_SEQ                | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| CROSS cvr_gp::{#cross_0#}           | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| INST \cvg_pkg::spi_cover_gp::cvr_gp | 100 | 100.00... | 100.00% |         | ✓ |         | 0 |
| CVP rx_data_all                     | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| bin wr_add                          | 1   | 100.00... | 14282   |         | ✓ |         |   |
| bin wr_data                         | 1   | 100.00... | 1716    |         | ✓ |         |   |
| bin read_address                    | 1   | 100.00... | 2274    |         | ✓ |         |   |
| bin read_data                       | 1   | 100.00... | 1728    |         | ✓ |         |   |
| CVP SS_n_seq                        | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| bin seq_                            | 1   | 100.00... | 469     |         | ✓ |         |   |
| bin seq2_                           | 1   | 100.00... | 143     |         | ✓ |         |   |
| bin start_transaction               | 1   | 100.00... | 612     |         | ✓ |         |   |
| CVP MOSI_SEQ                        | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| bin write_address                   | 1   | 100.00... | 12030   |         | ✓ |         |   |
| bin write_data                      | 1   | 100.00... | 1223    |         | ✓ |         |   |
| bin read_address                    | 1   | 100.00... | 1165    |         | ✓ |         |   |
| bin read_data                       | 1   | 100.00... | 1070    |         | ✓ |         |   |
| CROSS #cross_0#                     | 100 | 100.00... | 100.00% |         | ✓ |         |   |
| bin start_wrdata                    | 1   | 100.00... | 155     |         | ✓ |         |   |
| bin start_rdadd                     | 1   | 100.00... | 161     |         | ✓ |         |   |
| bin start_rddata                    | 1   | 100.00... | 143     |         | ✓ |         |   |
| bin seq_wradd                       | 1   | 100.00... | 308     |         | ✓ |         |   |



# SPI\_ASSERTIONS COVERAGE

|                                    |            |     |    |   |       |   |     |      |           |       |     |                                          |   |
|------------------------------------|------------|-----|----|---|-------|---|-----|------|-----------|-------|-----|------------------------------------------|---|
| + /top/DUT/ASSRT_rst_n_prop        | Concurrent | SVA | on | 0 | 10190 | 0 | 0B  | 160B | 30 ns     | 10190 | off | assert( @(posedge clk) (~rst_n) =...     | ✓ |
| + /top/DUT/ASSRT_VALID_wr_add_seq  | Concurrent | SVA | on | 0 | 60    | 1 | 80B | 80B  | 200030 ns | 599   | off | assert( @(posedge clk) disable iff (...) | ✓ |
| + /top/DUT/ASSRT_VALID_wr_data_seq | Concurrent | SVA | on | 0 | 41    | 1 | 80B | 160B | 203510 ns | 599   | off | assert( @(posedge clk) disable iff (...) | ✓ |
| + /top/DUT/ASSRT_VALID_rd_add_seq  | Concurrent | SVA | on | 0 | 46    | 1 | 80B | 160B | 205870 ns | 599   | off | assert( @(posedge clk) disable iff (...) | ✓ |
| + /top/DUT/ASSRT_VALID_rd_data_seq | Concurrent | SVA | on | 0 | 44    | 1 | 80B | 240B | 288350 ns | 599   | off | assert( @(posedge clk) disable iff (...) | ✓ |
| + /top/DUT/ASSRT_first             | Concurrent | SVA | on | 0 | 760   | 0 | 0B  | 80B  | 200030 ns | 760   | off | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| + /top/DUT/ASSRT_sec               | Concurrent | SVA | on | 0 | 742   | 0 | 0B  | 80B  | 200050 ns | 742   | off | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| + /top/DUT/ASSRT_third             | Concurrent | SVA | on | 0 | 304   | 0 | 0B  | 80B  | 200570 ns | 304   | off | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| + /top/DUT/ASSRT_forth             | Concurrent | SVA | on | 0 | 179   | 0 | 0B  | 80B  | 200290 ns | 179   | off | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| + /top/DUT/ASSRT_fifth             | Concurrent | SVA | on | 0 | 94    | 0 | 0B  | 80B  | 201610 ns | 94    | off | assert( disable iff (~rst_n) ( @(pos...  | ✓ |

| Branches - by instance (/top/DUT) |                                                       | Branch | ✓ | ✗ | E |
|-----------------------------------|-------------------------------------------------------|--------|---|---|---|
| 35                                | case (cs)                                             |        | ✓ |   |   |
| 36                                | IDLE: begin                                           |        | ✓ |   |   |
| 37                                | if (SS_n) ns = IDLE;                                  |        | ✓ |   |   |
| 38                                | else ns = CHK_CMD;                                    |        | ✓ |   |   |
| 40                                | CHK_CMD: begin                                        |        | ✓ |   |   |
| 41                                | if (SS_n) ns = IDLE;                                  |        | ✓ |   |   |
| 42                                | else begin                                            |        | ✓ |   |   |
| 43                                | if (~MOSI) ns = WRITE;                                |        | ✓ |   |   |
| 44                                | else begin                                            |        | ✓ |   |   |
| 45                                | if (received_address) ns = READ_DATA;                 |        | ✓ |   |   |
| 46                                | else ns = READ_ADD;                                   |        | ✓ |   |   |
| 50                                | WRITE: begin                                          |        | ✓ |   |   |
| 51                                | if (SS_n) ns = IDLE;                                  |        | ✓ |   |   |
| 52                                | else ns = WRITE;                                      |        | ✓ |   |   |
| 54                                | READ_ADD: begin                                       |        | ✓ |   |   |
| 55                                | if (SS_n) ns = IDLE;                                  |        | ✓ |   |   |
| 56                                | else ns = READ_ADD;                                   |        | ✓ |   |   |
| 58                                | READ_DATA: begin                                      |        | ✓ |   |   |
| 59                                | if (SS_n) ns = IDLE;                                  |        | ✓ |   |   |
| 60                                | else ns = READ_DATA;                                  |        | ✓ |   |   |
| 66                                | if (~rst_n) begin                                     |        | ✓ |   |   |
| 71                                | end else begin                                        |        | ✓ |   |   |
| 73                                | IDLE: begin                                           |        | ✓ |   |   |
| 78                                | CHK_CMD: begin                                        |        | ✓ |   |   |
| 81                                | WRITE: begin                                          |        | ✓ |   |   |
| 82                                | if (counter > 0) begin //this means it needs to write |        | ✓ |   |   |
| 85                                | end else rx_valid <= 1;                               |        | ✓ |   |   |
| 87                                | READ_ADD: begin                                       |        | ✓ |   |   |
| 88                                | if (counter > 0) begin                                |        | ✓ |   |   |
| 91                                | end else begin                                        |        | ✓ |   |   |
| 96                                | READ_DATA: begin                                      |        | ✓ |   |   |
| 97                                | if (tx_valid) begin                                   |        | ✓ |   |   |
| 99                                | if (counter > 0) begin                                |        | ✓ |   |   |
| 102                               | end else begin                                        |        | ✓ |   |   |
| 105                               | end else begin                                        |        | ✓ |   |   |
| 106                               | if (counter > 0 && ~rx_valid) begin                   |        | ✓ |   |   |
| 109                               | end else begin                                        |        | ✓ |   |   |
| 115                               | default:begin                                         |        | ✓ |   |   |

# SPI\_COVERAGE

Statements - by instance (/top/DUT)

Statement

✓

✗

E

✓

40 else ns = CHK\_CMD;

✓

41 if (SS\_n) ns = IDLE;

✓

43 if (~MOSI) ns = WRITE;

✓

45 if (received\_address) ns = READ\_DATA;

✓

46 else ns = READ\_ADD;

✓

51 if (SS\_n) ns = IDLE;

✓

52 else ns = WRITE;

✓

55 if (SS\_n) ns = IDLE;

✓

56 else ns = READ\_ADD;

✓

59 if (SS\_n) ns = IDLE;

✓

60 else ns = READ\_DATA;

✓

65 always @(posedge clk) begin

✓

67 rx\_data <= 0;

✓

68 rx\_valid <= 0;

✓

69 received\_address <= 0;

✓

70 MISO <= 0;

✓

74 rx\_valid <= 0;

✓

75 rx\_data<=0;

✓

76 MISO<=0;

✓

79 counter <= 10; //making counter =10 as if counter=0 then rx\_valid is 1

✓

83 rx\_data[counter-1] <= MOSI; //assign if write writing address or data

✓

84 counter <= counter - 1;

✓

85 end else rx\_valid <= 1;

✓

89 rx\_data[counter-1] <= MOSI;

✓

90 counter <= counter - 1;

✓

92 rx\_valid <= 1;

✓

93 received\_address <= 1;

✓

98 rx\_valid <= 0;

✓

100 MISO <= tx\_data[counter-1];

✓

101 counter <= counter - 1;

✓

103 received\_address <= 0;

✓

107 rx\_data[counter-1] <= MOSI;

✓

108 counter <= counter - 1;

✓

110 rx\_valid <= 1;

✓

111 counter <= 8;

✓

116 rx\_valid <= 0;

✗

117 rx\_data<=0;

✗

118 MISO<=0;

Toggles - by instance (/top/DUT)

sim:/top/DUT

✓ clk

+ ✓ counter

+ ✓ cs

✓ MISO

✓ MOSI

+ ✓ ns

✓ received\_address

✓ rst\_n

+ ✓ rx\_data

✓ rx\_valid

✓ SS\_n

+ ✓ tx\_data

✓ tx\_valid

FSMs - by instance (/top/DUT)

CS

+ ✓ IDLE (0)

+ ✓ CHK\_CMD (2)

+ ✓ READ\_ADD (3)

+ ✓ READ\_DATA (4)

+ ✓ WRITE (1)

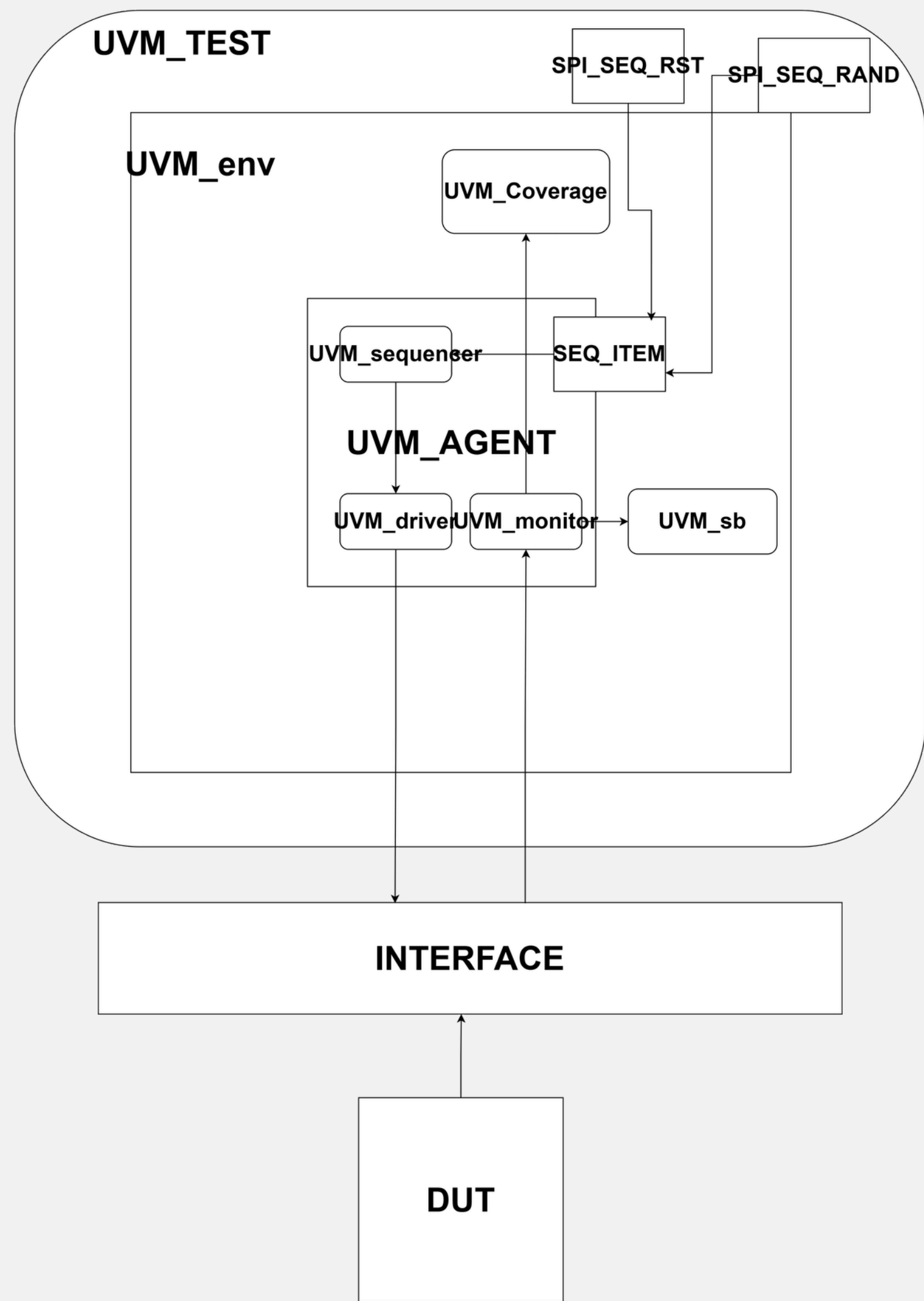
**COVER REPORT ON GITHUB**

**Excluding last of statement coverage as these are in default case and due to our constraints they can't occur**

# SPI\_UVM

```
"
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test spi_test...
# UVM_INFO spi_test.sv(30) @ 0: uvm_test_top [run_phase] rannd_begin
# *****
# * Questa UVM Transaction Recording Turned ON.                                *
# * recording_detail has been set.                                             *
# * To turn off, set 'recording_detail' to off:                               *
# * uvm_config_db#(int)                ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# *****
# UVM_INFO spi_test.sv(32) @ 200: uvm_test_top [run_phase] rst_end
# UVM_INFO spi_test.sv(33) @ 200: uvm_test_top [run_phase] rannd_begin
# UVM_INFO spi_test.sv(35) @ 200200: uvm_test_top [run_phase] rannd_end
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 200200: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO spi_sb.sv(47) @ 200200: uvm_test_top.spi_env_comp.sb [report_phase] Total correct:      10010
# UVM_INFO spi_sb.sv(48) @ 200200: uvm_test_top.spi_env_comp.sb [report_phase] Total false:       0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    10
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]    2
# [RNTST]         1
# [TEST_DONE]     1
# [report_phase]  2
# [run_phase]     4
```

# SPI\_DIAGRAM





# SPI\_plan

| LABEL | DESIGN REQUIREMENT DESCRIPTION                                              | STIMULUS GENERATION                                                                               | FUNCTIONAL COVERAGE                                                                                                                                                | FUNCTIONALITY CHECK                                                                                          |
|-------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| SPI_0 | when rst_n is 0 outputs must be equal zero                                  | a sequence is made at the start of UVM                                                            | -                                                                                                                                                                  | rst assertion is made also by comparing to a golden model                                                    |
| SPI_1 | When SS_n is 0 and MOSI becomes 0 0 0 a write address operation must occure | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when wr_data seq happens a bin is made to insure that a wr_add seq is made also at the same time a SS_n is made to ensure that SS_n is low for 13 clk cycle        | VALID_wr_add_seq assertion is made to insure that it works correctly<br>also comparing to a golden model     |
| SPI_2 | When SS_n is 0 and MOSI becomes 0 0 1 a write data operation must occure    | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when wr_data seq happens a bin is made to insure that a wr_data seq is made also at the same time a SS_n is made to ensure that SS_n is low for 13 clk cycle       | a VALID_wr_data _seq assertion is made to insure that it works correctly<br>also comparing to a golden model |
| SPI_3 | When SS_n is 0 and MOSI becomes 1 1 0 a read address operation must occure  | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when rd_address seq happens a bin is made to insure that a rd_address seq is made also at the same time a SS_n is made to ensure that SS_n is low for 13 clk cycle | VALID_rd_address_seq assertion is made to insure that it works correctly<br>also comparing to a golden model |
| SPI_4 | When SS_n is 0 and MOSI becomes 1 1 1 a read data operation must occure     | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when rd_data seq happens a bin is made to insure that a rd_data seq is made also at the same time a SS_n is made to ensure that SS_n is low for 23 clk cycle       | VALID_rd_data_seq assertion is made to insure that it works correctly<br>also comparing to a golden model    |



# PART\_2 RAM

## CODE BUGS

### 1-WRONG MEM[ADDRESS]

```
2'b11 : dout <= MEM[Wr_Addr];
```

### FIX

```
2'b11:   dout <= MEM[Rd_Addr]; //bug
```

### 2-tx\_valid wrong handling

```
end  
tx_valid <= (din[9] && din[8] && rx_valid)? 1'b1 : 1'b0;
```

### FIX

```
tx_valid <= (din[9] && din[8] && rx_valid && rst_n) ? 1'b1 : 1'b0;  
//correct tx_valid handle rst_n must be handled if rst_n output =0;
```

# RAM\_MODULE

```
1  module RAM (  
2      din,  
3      clk,  
4      rst_n,  
5      rx_valid,  
6      dout,  
7      tx_valid  
8  );  
9  
10     input [9:0] din;  
11     input clk, rst_n, rx_valid;  
12  
13     output reg [7:0] dout;  
14     output reg tx_valid;  
15  
16     reg [7:0] MEM[255:0];  
17  
18     reg [7:0] Rd_Addr, Wr_Addr;  
19  
20     always @(posedge clk) begin  
21         if (~rst_n) begin  
22             dout <= 0;  
23             Rd_Addr <= 0;  
24             Wr_Addr <= 0;  
25         end else if (rx_valid) begin  
26             case (din[9:8])  
27                 2'b00: Wr_Addr <= din[7:0];  
28                 2'b01: MEM[Wr_Addr] <= din[7:0];  
29                 2'b10: Rd_Addr <= din[7:0];  
30                 2'b11: dout <= MEM[Rd_Addr]; //bug  
31                 default: dout <= 0;  
32             endcase  
33         end  
34         tx_valid <= (din[9] && din[8] && rx_valid && rst_n) ? 1'b1 : 1'b0;  
35         //correct tx_valid handle  
36     end
```

# RAM\_IF\_conflg

```
1 package config_pkg;
2     import uvm_pkg::*;
3     `include "uvm_macros.svh"
4     class ram_config_obj extends uvm_object;
5         `uvm_object_utils(ram_config_obj);
6         virtual ramif ram_config_vif;
7         function new(string name = "ram_config");
8             super.new(name);
9         endfunction //new()
10    endclass
11 endpackage
12
```

```
1 interface ramif (
2     |   clk
3 );
4     input clk;
5     logic [9:0] din;
6     logic rst_n, rx_valid;
7     logic [7:0] dout, dout_exp;
8     logic tx_valid, tx_valid_exp;
9 endinterface
10
```

# RAM\_TOP

```
1 import uvm_pkg::*;
2 `include "uvm_macros.svh"
3 import ram_test_pkg::*;
4 import config_pkg::*;
5 module top ();
6     bit clk;
7     initial begin
8         clk = 0;
9         forever begin
10             #10 clk = ~clk;
11         end
12     end
13     ramif ramif_ (clk);
14     RAM DUT (
15         ramif_.din,
16         clk,
17         ramif_.rst_n,
18         ramif_.rx_valid,
19         ramif_.dout,
20         ramif_.tx_valid
21     );
22     RAM_GM RAM_GM_ (
23         ramif_.din,
24         clk,
25         ramif_.rst_n,
26         ramif_.rx_valid,
27         ramif_.dout_exp,
28         ramif_.tx_valid_exp
29     );
30     initial begin
31         uvm_config_db#(virtual ramif)::set(null, "uvm_test_top", "KEY", ramif_);
32         run_test("ram_test");
33     end
```

```
34 bind RAM ram_SVA sva_instance (
35     ramif_.din,
36     clk,
37     ramif_.rst_n,
38     ramif_.rx_valid,
39     ramif_.dout,
40     ramif_.tx_valid
41 );
42 endmodule
43
```

# RAM\_TEST

```
1 package ram_test_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import ram_env_pkg::*;
5 import config_pkg::*;
6 import seq_rand_pkg::*;
7 import seq_rst_pkg::*;
8 import seq_wr_only_pkg::*;
9 import seq_rd_only_pkg::*;
10 import seq_wr_rd_only_pkg::*;
11 class ram_test extends uvm_test;
12     `uvm_component_utils(ram_test)
13     ram_env ram_env_comp;
14     ram_config_obj ram_config_obj_test;
15     // seq_rand randd;
16     seq_rst rst;
17     seq_wr_only wr_only;
18     seq_rd_only rd_only;
19     seq_wr_rd_only wr_rd;
20     function new(string name = "ram_test", uvm_component parent = null);
21         super.new(name, parent);
22     endfunction
23     function void build_phase(uvm_phase phase);
24         super.build_phase(phase);
25         ram_env_comp = ram_env::type_id::create("ram_env_comp", this);
26         ram_config_obj_test = ram_config_obj::type_id::create("ram_config_comp", this);
27         // randd = seq_rand::type_id::create("randd");
28         rst = seq_rst::type_id::create("rst");
29         wr_only = seq_wr_only::type_id::create("wr_only");
30         rd_only = seq_rd_only::type_id::create("rd_only");
31         wr_rd = seq_wr_rd_only::type_id::create("wr_rd_only");
32         if (~uvm_config_db#(virtual ramif)::get(this, "", "KEY", ram_config_obj_test.ram_config_vif))
33             `uvm_fatal("build_phase", "Test -Unable to get into virtual interface");
34         uvm_config_db#(ram_config_obj)::set(this, "*", "KEY", ram_config_obj_test);
35     endfunction
```

```
36     task run_phase(uvm_phase phase);
37         super.run_phase(phase);
38         phase.raise_objection(this);
39         `uvm_info("run_phase", "begin ", UVM_LOW);
40         rst.start(ram_env_comp.agent.sqr);
41         wr_only.start(ram_env_comp.agent.sqr);
42         rd_only.start(ram_env_comp.agent.sqr);
43         wr_rd.start(ram_env_comp.agent.sqr);
44         `uvm_info("run_phase", "end", UVM_LOW);
45         phase.drop_objection(this);
46     endtask
47 endclass
48 endpackage
```



# RAM\_ENV

```
1  package ram_env_pkg;
2      import uvm_pkg::*;
3      import ram_driver_pkg::*;
4      import config_pkg::*;
5      import sequencer_pkg::*;
6      import ram_monitor_pkg::*;
7      import ram_seq_item_pkg::*;
8      import ram_agent_pkg::*;
9      import sb_pkg::*;
10     import cvg_pkg::*;
11     `include "uvm_macros.svh"
12     class ram_env extends uvm_env;
13     `uvm_component_utils(ram_env)
14     ram_agent agent;
15     ram_cover_gp cgp;
16     ram_seq_item seq_item;
17     ram_sb sb;
18     function new(string name = "ram_env", uvm_component parent = null);
19     |     super.new(name, parent);
20     endfunction //new()
21     function void build_phase(uvm_phase phase);
22     |     super.build_phase(phase);
23     |     agent = ram_agent::type_id::create("agent", this);
24     |     sb = ram_sb::type_id::create("sb", this);
25     |     cgp = ram_cover_gp::type_id::create("cgp", this);
26     endfunction
27     function void connect_phase(uvm_phase phase);
28     |     super.connect_phase(phase);
29     |     agent.agt_ap.connect(sb.sb_export);
30     |     agent.agt_ap.connect(cgp.cgp_export);
31     endfunction
32     endclass
33 endpackage
```

# RAM\_AGENT

```
1  package ram_agent_pkg;
2      import uvm_pkg::*;
3      import ram_driver_pkg::*;
4      import config_pkg::*;
5      import sequencer_pkg::*;
6      import ram_monitor_pkg::*;
7      import ram_seq_item_pkg::*;
8      `include "uvm_macros.svh"
9      class ram_agent extends uvm_agent;
10     `uvm_component_utils(ram_agent)
11     ram_sequencer sqr;
12     ram_driver drv;
13     ram_config_obj cnfg;
14     ram_monitor mon;
15     uvm_analysis_port #(ram_seq_item) agt_ap;
16     function new(string name = "ram_agent", uvm_component parent = null);
17     |   super.new(name, parent);
18     endfunction
19     function void build_phase(uvm_phase phase);
20     |   super.build_phase(phase);
21     |   agt_ap = new("agt_ap", this);
22     |   if (~uvm_config_db#(ram_config_obj)::get(this, "", "KEY", cnfg)) begin
23     |   |   `uvm_fatal("build_phase", "unable to conf");
24     |   end
25     |   sqr = ram_sequencer::type_id::create("sqr", this);
26     |   drv = ram_driver::type_id::create("drv", this);
27     |   mon = ram_monitor::type_id::create("mon", this);
28     endfunction
29     function void connect_phase(uvm_phase phase);
30     |   super.connect_phase(phase);
31     |   drv.ram_driver_vif = cnfg.ram_config_vif;
32     |   mon.ram_monitor_if = cnfg.ram_config_vif;
33     |   drv.seq_item_port.connect(sqr.seq_item_export);
34     |   mon.mon_ap.connect(agt_ap);
35     endfunction
36     endclass
37 endpackage
```

# RAM\_MONITOR\_DRIVER

```
1 package ram_monitor_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import ram_seq_item_pkg::*;
5 class ram_monitor extends uvm_monitor;
6     `uvm_component_utils(ram_monitor);
7     virtual ramif ram_monitor_if;
8     ram_seq_item seq_item;
9     uvm_analysis_port #(ram_seq_item) mon_ap;
10    function new(string name = "ram_monitor", uvm_component parent = null);
11        super.new(name, parent);
12    endfunction
13    function void build_phase(uvm_phase phase);
14        super.build_phase(phase);
15        mon_ap = new("mon_ap", this);
16    endfunction
17    task run_phase(uvm_phase phase);
18        super.run_phase(phase);
19        forever begin
20            seq_item = ram_seq_item::type_id::create("seq_item_ram");
21            //we must sample the outputs of the DUT
22            @(negedge ram_monitor_if.clk);
23            seq_item.din      = ram_monitor_if.din;
24            seq_item.rst_n    = ram_monitor_if.rst_n;
25            seq_item.rx_valid = ram_monitor_if.rx_valid;
26            seq_item.dout     = ram_monitor_if.dout;
27            seq_item.tx_valid = ram_monitor_if.tx_valid;
28            mon_ap.write(seq_item);
29            //also at the same time sample the outputs of the GOLDEN MODEL
30            seq_item.tx_valid_exp = ram_monitor_if.tx_valid_exp;
31            seq_item.dout_exp = ram_monitor_if.dout_exp;
32            `uvm_info("run_phase", seq_item.convert2string(), UVM_HIGH);
33        end
34    endtask
35 endclass
36 endpackage
```

```
1 package ram_driver_pkg;
2 import uvm_pkg::*;
3 import config_pkg::*;
4 import ram_seq_item_pkg::*;
5 `include "uvm_macros.svh"
6 class ram_driver extends uvm_driver #(ram_seq_item);
7     `uvm_component_utils(ram_driver)
8     virtual ramif ram_driver_vif;
9     ram_config_obj ram_config_obj_driver;
10    ram_seq_item seq_item;
11    function new(string name = "ram_driver", uvm_component parent = null);
12        super.new(name, parent);
13    endfunction //new()
14    task run_phase(uvm_phase phase);
15        super.run_phase(phase);
16        forever begin
17            seq_item = ram_seq_item::type_id::create("seq_item_driver");
18            seq_item_port.get_next_item(seq_item);
19            ram_driver_vif.din      = seq_item.din;
20            ram_driver_vif.rst_n    = seq_item.rst_n;
21            ram_driver_vif.rx_valid = seq_item.rx_valid;
22            @(negedge ram_driver_vif.clk);
23            seq_item_port.item_done();
24        end
25    endtask
26 endclass
27 endpackage
28
```

# RAM\_SEQ\_ITEM

```
1 package ram_seq_item_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 class ram_seq_item extends uvm_sequence_item;
5     `uvm_object_utils(ram_seq_item)
6     logic [9:0] din;
7     rand logic rst_n, rx_valid;
8     logic tx_valid;
9     logic [7:0] dout;
10    rand logic [7:0] data;
11    rand logic [1:0] seq;
12    logic wr_data, rd_data, rd_add_, wr_data_done;
13    logic [7:0] dout_exp;
14    logic tx_valid_exp;
15    function new(string name = "ram_seq_item");
16        super.new(name);
17        wr_data = 0;
18        wr_data_done = 0;
19        rd_add_ = 0;
20        rd_data = 0;
21    endfunction
```

```
15    function new(string name = "ram_seq_item");
16        super.new(name);
17        wr_data = 0;
18        wr_data_done = 0;
19        rd_add_ = 0;
20        rd_data = 0;
21    endfunction
22    constraint cons {
23        rst_n dist {
24            1 := 98,
25            0 := 2
26        };
27        rx_valid dist {
28            1 := 80,
29            0 := 20
30        };
31    }
32    constraint write_only_seq {seq inside {3'b000, 3'b001};}
33    constraint read_only_seq {
34        if (rd_add_)
35            seq inside {3'b111};
36        else
37            if (rd_data)
38                seq inside {3'b110};
39            else
40                seq inside {3'b111, 3'b110};
41    }
```

# RAM\_SEQ\_ITEM

```
56 function void post_randomize();
57     din = {seq, data};
58     if (seq == 2'b00) begin
59         wr_data = 1;
60         wr_data_done = 0;
61     end else if (seq == 2'b01) begin
62         wr_data = 0;
63         wr_data_done = 1;
64     end else begin
65         wr_data = 0;
66         wr_data_done = 0;
67     end //wr_handle
68     if (seq == 2'b10) begin
69         rd_add_ = 1;
70         rd_data = 0;
71     end else if (seq == 2'b11) begin
72         rd_data = 1;
73         rd_add_ = 0;
74     end else begin
75         rd_add_ = 0;
76         rd_data = 0;
77     end
78 endfunction
```

```
79 function string convert2string();
80     return $sformatf(
81         "%s din=%0d, rst_n=%0b,rx_valid=%0b, tx_valid=%0b, dout=%0b",
82         super.convert2string(),
83         din,
84         rst_n,
85         rx_valid,
86         tx_valid,
87         dout,
88     );
89 endfunction
90 function string convert2string_stimulus();
91     return $sformatf(
92         "din=%0d, rst_n=%0b,rx_valid=%0b, tx_valid=%0b, dout=%0b",
93         din,
94         rst_n,
95         rx_valid,
96         tx_valid,
97         dout,
98     );
99 endfunction
100 endclass
101 endpackage
102
```

```
constraint wr_rd_seq {
    if (wr_data) seq inside {3'b000, 3'b001};
    if (wr_data_done)
        seq dist {
            3'b000 := 40,
            3'b110 := 60
        };
    if (rd_add_) seq inside {3'b111};
    if (rd_data)
        seq dist {
            3'b000 := 60,
            3'b110 := 40
        };
}
```



# RAM\_SEQ\_RST| WR\_ONLY

```
1 package seq_rst_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import ram_seq_item_pkg::*;
5 class seq_rst extends uvm_sequence #(ram_seq_item);
6     `uvm_object_utils(seq_rst);
7     ram_seq_item seq_item;
8     function new(string name = "seq_rst");
9         super.new(name);
10    endfunction
11    task body;
12        seq_item = ram_seq_item::type_id::create("seq_item_rst");
13        repeat (10) begin
14            start_item(seq_item);
15            seq_item.rst_n = 0;
16            finish_item(seq_item);
17        end
18    endtask
19 endclass //seq_rst
20 endpackage
```

```
1 package seq_wr_only_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import ram_seq_item_pkg::*;
5 class seq_wr_only extends uvm_sequence #(ram_seq_item);
6     `uvm_object_utils(seq_wr_only);
7     ram_seq_item seq_item;
8     function new(string name = "seq_wr_only");
9         super.new(name);
10    endfunction
11    task body;
12        seq_item = ram_seq_item::type_id::create("seq_item_wr_only");
13        repeat (10000) begin
14            start_item(seq_item);
15            seq_item.write_only_seq.constraint_mode(1);
16            seq_item.read_only_seq.constraint_mode(0);
17            seq_item.wr_rd_seq.constraint_mode(0);
18            assert (seq_item.randomize());
19            finish_item(seq_item);
20        end
21    endtask
22 endclass //seq_wr_only
23 endpackage
24
```

# RAM\_RD\_ONLY |WR\_RD

```
1 package seq_rd_only_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import ram_seq_item_pkg::*;
5 class seq_rd_only extends uvm_sequence #(ram_seq_item);
6     `uvm_object_utils(seq_rd_only);
7     ram_seq_item seq_item;
8     function new(string name = "seq_rd_only");
9         super.new(name);
10    endfunction
11    task body;
12        seq_item = ram_seq_item::type_id::create("seq_item_rd_only");
13        repeat (10000) begin
14            start_item(seq_item);
15            seq_item.write_only_seq.constraint_mode(0);
16            seq_item.read_only_seq.constraint_mode(1);
17            seq_item.wr_rd_seq.constraint_mode(0);
18            assert (seq_item.randomize());
19            finish_item(seq_item);
20        end
21    endtask
22 endclass //seq_rd_only
23 endpackage
24
```

```
1 package seq_wr_rd_only_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import ram_seq_item_pkg::*;
5 class seq_wr_rd_only extends uvm_sequence #(ram_seq_item);
6     `uvm_object_utils(seq_wr_rd_only);
7     ram_seq_item seq_item;
8     function new(string name = "seq_wr_rd_only");
9         super.new(name);
10    endfunction
11    task body;
12        seq_item = ram_seq_item::type_id::create("seq_item_wr_rd_only");
13        repeat (10000) begin
14            start_item(seq_item);
15            seq_item.write_only_seq.constraint_mode(0);
16            seq_item.read_only_seq.constraint_mode(0);
17            seq_item.wr_rd_seq.constraint_mode(1);
18            assert (seq_item.randomize());
19            finish_item(seq_item);
20        end
21    endtask
22 endclass
23 endpackage
24
```

# RAM\_SQR

```
1  package sequencer_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4      import ram_seq_item_pkg::*;
5      import seq_rand_pkg::*;
6  class ram_sequencer extends uvm_sequencer #(ram_seq_item);
7      `uvm_component_utils(ram_sequencer)
8  function new(string name = "ram_sequencer", uvm_component parent = null);
9      super.new(name, parent);
10 endfunction
11 endclass
12 endpackage
13
```

# RAM\_SB

```
1 package sb_pkg;
2   import config_pkg::*;
3   import ram_seq_item_pkg::*;
4   import uvm_pkg::*;
5   import ram_agent_pkg::*;
6   import seq_rand_pkg::*;
7   `include "uvm_macros.svh"
8   class ram_sb extends uvm_scoreboard;
9     `uvm_component_utils(ram_sb);
10    uvm_analysis_export #(ram_seq_item) sb_export;
11    uvm_tlm_analysis_fifo #(ram_seq_item) sb_fifo;
12    ram_seq_item seq_item;
13    int error_count = 0;
14    int correct_count = 0;
15    function new(string name = "scoreboard", uvm_component parent = null);
16      super.new(name, parent);
17    endfunction
18    function void build_phase(uvm_phase phase);
19      super.build_phase(phase);
20      sb_export = new("sb_export", this);
21      sb_fifo = new("sb_fifo", this);
22    endfunction
23    function void connect_phase(uvm_phase phase);
24      super.connect_phase(phase);
25      sb_export.connect(sb_fifo.analysis_export);
26    endfunction
27    task run_phase(uvm_phase phase);
28      super.run_phase(phase);
```

```
29      forever begin
30        sb_fifo.get(seq_item);
31        if (seq_item.dout != seq_item.dout_exp || seq_item.tx_valid != seq_item.tx_valid_exp) begin
32          `uvm_error("run_phase", $sformatf(
33            "comparason failed,DUT_transaction=%s dout_ref=0b%0b refrence tx_valid=0b%0b",
34            seq_item.convert2string(),
35            seq_item.dout_exp,
36            seq_item.tx_valid_exp,
37          ));
38          error_count = error_count + 1;
39        end else begin
40          correct_count = correct_count + 1;
41        end
42      end
43    endtask
44    function void report_phase(uvm_phase phase);
45      super.report_phase(phase);
46      `uvm_info("report_phase", $sformatf("Total correct:%d", correct_count), UVM_MEDIUM);
47      `uvm_info("report_phase", $sformatf("Total false:%d", error_count), UVM_MEDIUM);
48    endfunction
49  endclass
50 endpackage
```

# RAM\_GM

```
1  module RAM_GM (  
2      din,  
3      clk,  
4      rst_n,  
5      rx_valid,  
6      dout,  
7      tx_valid  
8  );  
9      parameter MEM_DEPTH = 256;  
10     parameter ADDR_SIZE = 8;  
11     input [9:0] din;  
12     input clk, rx_valid, rst_n;  
13     output reg [7:0] dout;  
14     output reg tx_valid;  
15     reg [7:0] read_address, write_address;  
16     reg [ADDR_SIZE-1:0] mem[MEM_DEPTH-1:0];
```

```
17     always @(posedge clk) begin  
18         if (~rst_n) begin  
19             dout <= 0;  
20             tx_valid <= 0;  
21             write_address <= 0;  
22             read_address <= 0;  
23         end else begin  
24             if (rx_valid) begin  
25                 if (din[9:8] == 2'b00) begin  
26                     write_address <= din[7:0];  
27                     tx_valid <= 0;  
28                 end else if (din[9:8] == 2'b01) begin  
29                     mem[write_address] <= din[7:0];  
30                     tx_valid <= 0;  
31                 end else if (din[9:8] == 2'b10) begin  
32                     read_address <= din[7:0];  
33                     tx_valid <= 0;  
34                 end else if (din[9:8] == 2'b11) begin  
35                     tx_valid <= 1;  
36                     dout <= mem[read_address];  
37                 end  
38             end else begin  
39                 tx_valid <= 0;  
40             end  
41         end  
42     end  
43 endmodule
```



# RAM\_SVA

```
1 module ram_SVA (  
2     din,  
3     clk,  
4     rst_n,  
5     rx_valid,  
6     dout,  
7     tx_valid  
8 );  
9     input [9:0] din;  
10    input clk, rx_valid, rst_n;  
11    input reg [7:0] dout;  
12    input reg tx_valid;  
13    property rst_n_prop;  
14        @(posedge clk) !rst_n | => {~tx_valid && ~dout};  
15    endproperty  
16    property assertion_2_tx_valid_low;  
17        @(posedge clk)  
18        disable iff(!rst_n && !rx_valid)  
19        @(posedge clk) (din[9:8] inside {2'b00, 2'b01, 2'b10}) | => ~tx_valid;  
20    endproperty  
21    property tx_valid_high;  
22        @(posedge clk)  
23        disable iff(!rst_n)  
24        @(posedge clk) (din[9:8] inside {2'b11}) | => tx_valid | => $fell(tx_valid)[->1]);  
25    endproperty  
26    property wr_seq;  
27        @(posedge clk)  
28        disable iff(!rst_n && !rx_valid)  
29        (din[9:8] inside {2'b00}) | -> ##[1:$] (din[9:8] inside {2'b01});  
30    endproperty  
31    property rd_add_data;  
32        @(posedge clk)  
33        disable iff(!rst_n && !rx_valid)  
34        (din[9:8] inside {2'b10}) | -> ##[1:$] (din[9:8] inside {2'b11});  
35    endproperty
```

```
26    property wr_seq;  
27        @(posedge clk)  
28        disable iff(!rst_n && !rx_valid)  
29        (din[9:8] inside {2'b00}) | -> ##[1:$] (din[9:8] inside {2'b01});  
30    endproperty  
31    property rd_add_data;  
32        @(posedge clk)  
33        disable iff(!rst_n && !rx_valid)  
34        (din[9:8] inside {2'b10}) | -> ##[1:$] (din[9:8] inside {2'b11});  
35    endproperty  
36    property rd_data_add;  
37        @(posedge clk)  
38        disable iff(!rst_n && !rx_valid)  
39        (din[9:8] inside {2'b11}) | -> ##[1:$] (din[9:8] inside {2'b10});  
40    endproperty  
41    assert_rst_n_prop: assert property (rst_n_prop);  
42    cover_rst_n_prop: cover property (rst_n_prop);  
43    assert_assertion_2_tx_valid_low: assert property property assertion_2_tx_valid_low;  
44    cover_assertion_2_tx_valid_low: cover property (assertion_2_tx_valid_low);  
45    assert_tx_valid_high: assert property (tx_valid_high);  
46    cover_tx_valid_high: cover property (tx_valid_high);  
47    assert_rd_add_data: assert property (rd_add_data);  
48    cover_rd_add_data: cover property (rd_add_data);  
49    assert_rd_data_add: assert property (rd_data_add);  
50    cover_rd_data_add: cover property (rd_data_add);  
51 endmodule  
52
```

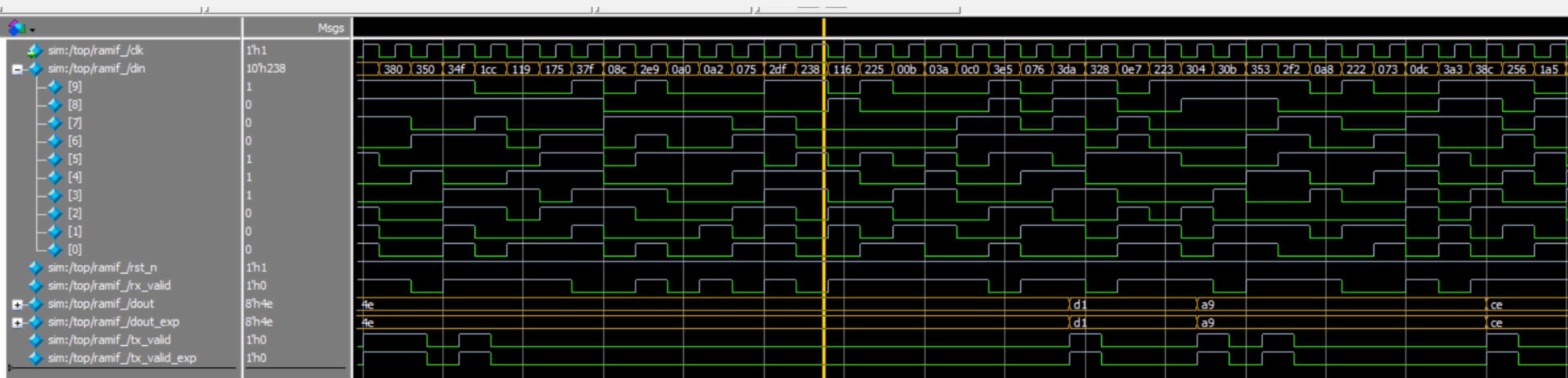
# RAM\_SVA

| ASSERTION_NAME                  | DESCRIPTION                                                                                                                       | ASSERTION                                                                                                          |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>rst_n_prop</b>               | When reset is asserted, tx_valid and dout should be cleared on next clock cycle                                                   | <pre>@(posedge clk) !rst_n ==&gt; {~tx_valid &amp;&amp; ~dout}</pre>                                               |
| <b>assertion_2_tx_valid_low</b> | For commands 00, 01, 10 (non-read-data), tx_valid should be low in the next cycle (disabled during reset or when rx_valid is low) | <pre>disable iff(!rst_n&amp;&amp;!rx_valid) (din[9:8] inside {2'b00,2'b01,2'b10} ==&gt; ~tx_valid)</pre>           |
| <b>tx_valid_high</b>            | For command 11 (read data), tx_valid should go high and then eventually fall (disabled during reset)                              | <pre>disable iff(!rst_n) (din[9:8] inside {2'b11} ==&gt; tx_valid ==&gt; \$fell(tx_valid)[-&gt;1])</pre>           |
| <b>wr_seq</b>                   | Write sequence: after address (00), eventually data (01) should follow (disabled during reset or when rx_valid is low)            | <pre>disable iff(!rst_n&amp;&amp;!rx_valid) (din[9:8] inside{2'b00}) -&gt; ##[1:\$] (din[9:8] inside{2'b01})</pre> |
| <b>rd_add_data</b>              | Read sequence: after address (10), eventually data (11) should follow (disabled during reset or when rx_valid is low)             | <pre>disable iff(!rst_n&amp;&amp;!rx_valid) (din[9:8] inside{2'b10}) -&gt; ##[1:\$] (din[9:8] inside{2'b11})</pre> |

# RAM\_PLAN

| LABEL       | DESIGN REQUIREMENT DESCRIPTION                                                                                                                                                                                                                                                                   | STIMULUS GENERATION                                                                                    | FUNCTIONAL COVERAGE                                                 | FUNCTIONALITY CHECK                                       |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------|
| RAM_RST_SEQ | when rst_n is 0 outputs must be equal zero                                                                                                                                                                                                                                                       | a seq that make the rst_n=0                                                                            | -                                                                   | rst assertion is made also by comparing to a golden model |
| WR_ONLY_SEQ | wr_only is a seq inside 000 001 and make the ram either write address or data                                                                                                                                                                                                                    | by turning on and off a certain constraint and that constraint is a write_only_seq it includes 000 001 | bins transitions are made to insure that wr_address or data happens | comparing with golden model and by assertions             |
| RD_ONLY_SEQ | every Read Address operation shall always be followed by Read Data. After a Read Data operation shall always be followed by Read Address.                                                                                                                                                        | by turning on and off constraints and only constraint of RD_ONLY_SEQ is on                             | transition bins are made to insure that the wanted seq occurred     | comparing with golden model and by assertions             |
| WR_RD_SEQ   | A Write Address is always followed by either another Write Address or Write Data. After Write Data, the next operation is 60% Read Address and 40% Write Address. A Read Address is always followed by Read Data. After Read Data, the next operation is 60% Write Address and 40% Read Address. | turning off all constraints except WR_RD_ONLY_SEQ                                                      | transition bins are made to insure that the wanted seq occurred     | comparing with golden model and by assertions             |

# RAM\_WAVEFORM





# RAM\_ASSERTION

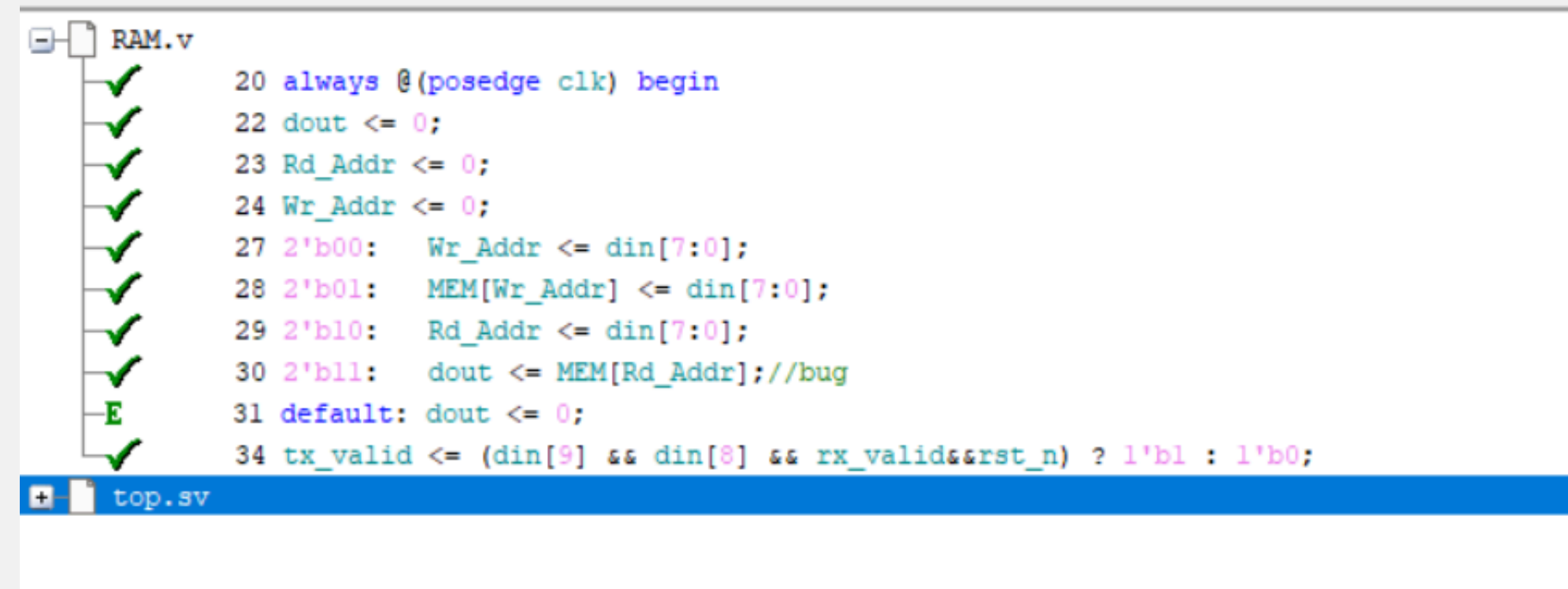
| Name                                                     | Language | Enabled | Log | Count | AtLeast | Limit   | Weight | Cmplt % | Cmplt graph                                                                         | Included |
|----------------------------------------------------------|----------|---------|-----|-------|---------|---------|--------|---------|-------------------------------------------------------------------------------------|----------|
| ▲ /top/DUT/sva_instance/cover_rst_n_prop                 | SVA      | ✓       | Off | 615   | 1       | Unli... | 1      | 100%    |  | ✓        |
| ▲ /top/DUT/sva_instance/cover_assertion_2_tx_valid_lo... | SVA      | ✓       | Off | 22395 | 1       | Unli... | 1      | 100%    |  | ✓        |
| ▲ /top/DUT/sva_instance/cover_tx_valid_high              | SVA      | ✓       | Off | 5620  | 1       | Unli... | 1      | 100%    |  | ✓        |
| ▲ /top/DUT/sva_instance/cover_rd_add_data                | SVA      | ✓       | Off | 7335  | 1       | Unli... | 1      | 100%    |  | ✓        |
| ▲ /top/DUT/sva_instance/cover_rd_data_add                | SVA      | ✓       | Off | 7271  | 1       | Unli... | 1      | 100%    |  | ✓        |

# RAM\_FUNCTIONAL COVERAGE

|                     |  |  |  |     |           |         |                                                                                       |   |         |
|---------------------|--|--|--|-----|-----------|---------|---------------------------------------------------------------------------------------|---|---------|
| /cvg_pkg/ram_cov... |  |  |  | 100 | 100.00... | 100.00% |   | ✓ | auto(1) |
| TYPE cvr_gp         |  |  |  | 100 | 100.00... | 100.00% |  | ✓ |         |
| CVP cvr_g...        |  |  |  | 100 | 100.00... | 100.00% |  | ✓ |         |
| bin wr_...          |  |  |  | 1   | 100.00... | 7535    |  | ✓ |         |
| bin wr_...          |  |  |  | 1   | 100.00... | 7543    |  | ✓ |         |
| bin read...         |  |  |  | 1   | 100.00... | 7502    |  | ✓ |         |
| bin read...         |  |  |  | 1   | 100.00... | 7420    |  | ✓ |         |
| bin tran...         |  |  |  | 1   | 100.00... | 7543    |  | ✓ |         |
| bin tran...         |  |  |  | 1   | 100.00... | 7543    |  | ✓ |         |
| bin tran...         |  |  |  | 1   | 100.00... | 7543    |  | ✓ |         |
| CVP cvr_g...        |  |  |  | 100 | 100.00... | 100.00% |  | ✓ |         |
| bin rx_v...         |  |  |  | 1   | 100.00... | 5894    |  | ✓ |         |
| bin rx_v...         |  |  |  | 1   | 100.00... | 24106   |  | ✓ |         |
| CVP cvr_g...        |  |  |  | 100 | 100.00... | 100.00% |  | ✓ |         |
| bin tx_v...         |  |  |  | 1   | 100.00... | 24141   |  | ✓ |         |
| bin tx_v...         |  |  |  | 1   | 100.00... | 5869    |  | ✓ |         |
| CROSS cvr...        |  |  |  | 100 | 100.00... | 100.00% |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 6036    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 1507    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 6036    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 1507    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 6036    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 1507    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 5973    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 1447    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 6055    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 1447    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 6036    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 1507    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 6042    |  | ✓ |         |
| bin <rx...          |  |  |  | 1   | 100.00... | 1493    |  | ✓ |         |
| CROSS cvr...        |  |  |  | 100 | 100.00... | 100.00% |  | ✓ |         |
| bin dinX...         |  |  |  | 1   | 100.00... | 5869    |  | ✓ |         |



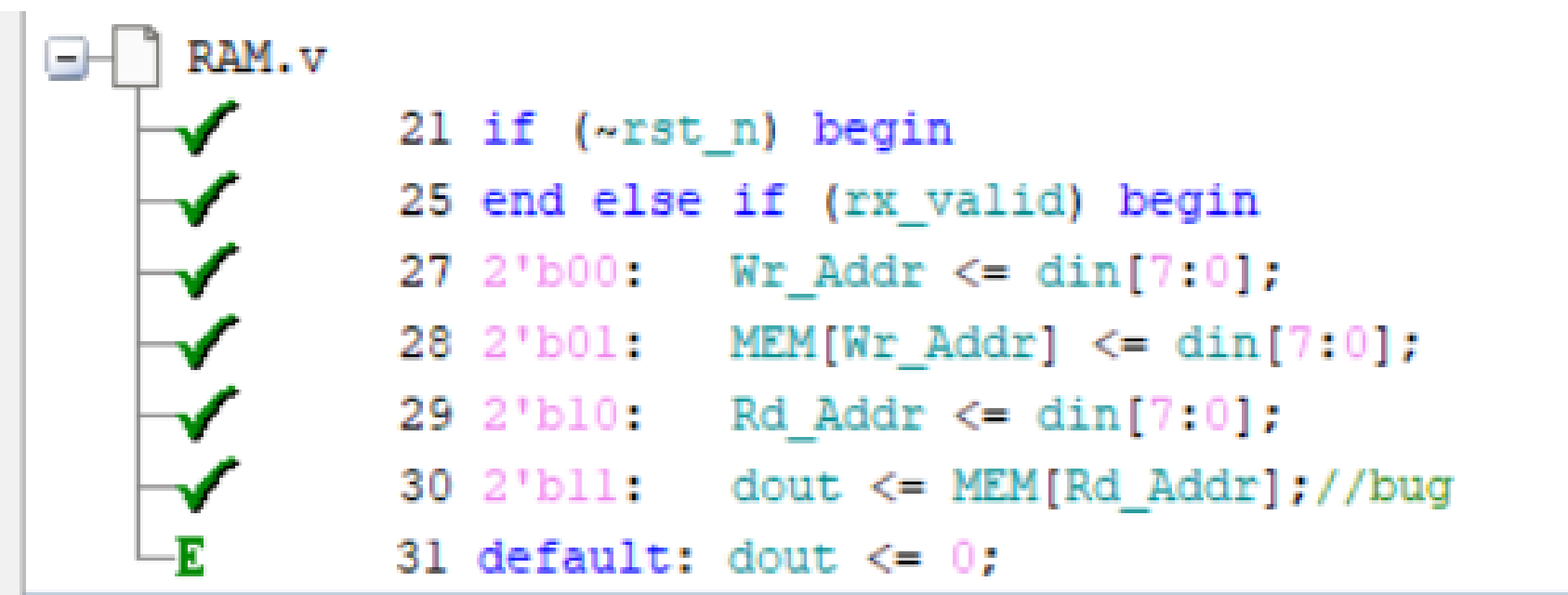
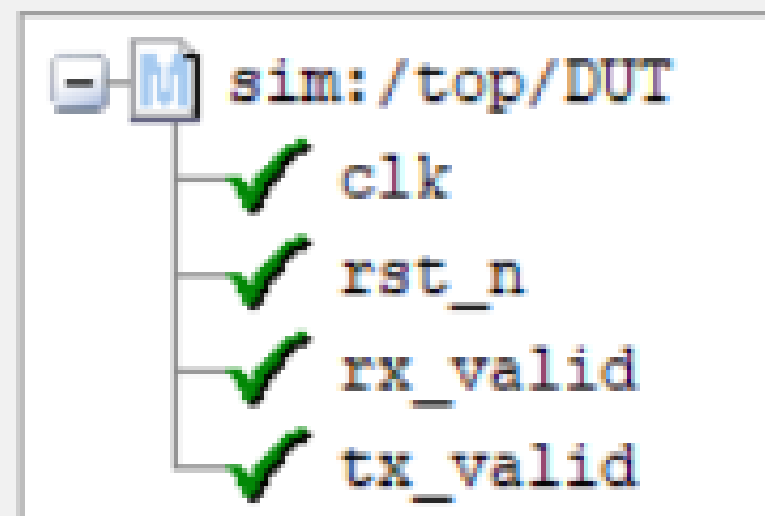
# RAM\_COVERAGE



RAM.v

```
20 always @(posedge clk) begin
22 dout <= 0;
23 Rd_Addr <= 0;
24 Wr_Addr <= 0;
27 2'b00: Wr_Addr <= din[7:0];
28 2'b01: MEM[Wr_Addr] <= din[7:0];
29 2'b10: Rd_Addr <= din[7:0];
30 2'b11: dout <= MEM[Rd_Addr]; //bug
31 default: dout <= 0;
34 tx_valid <= (din[9] && din[8] && rx_valid && rst_n) ? 1'b1 : 1'b0;
```

top.sv



RAM.v

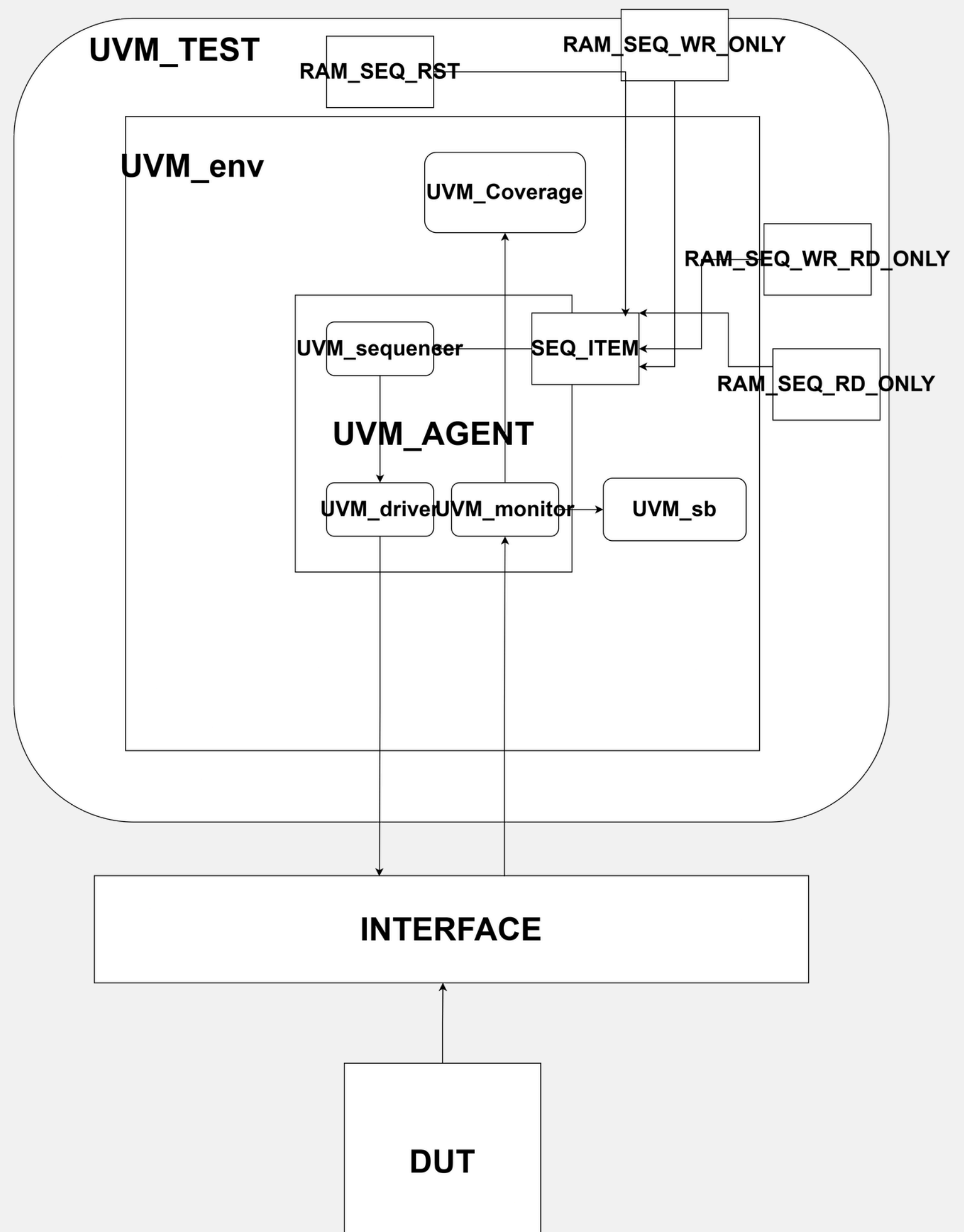
```
21 if (~rst_n) begin
25 end else if (rx_valid) begin
27 2'b00: Wr_Addr <= din[7:0];
28 2'b01: MEM[Wr_Addr] <= din[7:0];
29 2'b10: Rd_Addr <= din[7:0];
30 2'b11: dout <= MEM[Rd_Addr]; //bug
31 default: dout <= 0;
```

**Excluding default as it is impossible to hit**

# RAM\_UVM

```
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test ram_test...
# UVM_INFO ram_test.sv(39) @ 0: uvm_test_top [run_phase] begin
# *****
# * Questa UVM Transaction Recording Turned ON.                                *
# * recording_detail has been set.                                              *
# * To turn off, set 'recording_detail' to off:                                *
# * uvm_config_db#(int)                ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# *****
# UVM_INFO ram_test.sv(44) @ 600200: uvm_test_top [run_phase] end
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 600200: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ram_sb.sv(46) @ 600200: uvm_test_top.ram_env_comp.sb [report_phase] Total correct:      30010
# UVM_INFO ram_sb.sv(47) @ 600200: uvm_test_top.ram_env_comp.sb [report_phase] Total false:      0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :      8
# UVM_WARNING :    0
# UVM_ERROR :     0
# UVM_FATAL :     0
# ** Report counts by id
# [Questa UVM]      2
# [RNTST]           1
# [TEST_DONE]       1
# [report_phase]    2
# [run_phase]       2
# ** Note: $finish      : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#   Time: 600200 ns  Iteration: 61  Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

# RAM\_DIAGRAM



# PART\_3 WRAPPER

CODE BUGS

NO BUGS

# WRAPPER MODULE/ WRAPPER GM

```
1 module WRAPPER (  
2     MOSI,  
3     SS_n,  
4     clk,  
5     rst_n,  
6     MISO  
7 );  
8  
9 input MOSI, SS_n, clk, rst_n;  
10 output MISO;  
11  
12 wire [9:0] rx_data_din;  
13 wire      rx_valid;  
14 wire      tx_valid;  
15 wire [7:0] tx_data_dout;  
16  
17 RAM RAM_instance (  
18     rx_data_din,  
19     clk,  
20     rst_n,  
21     rx_valid,  
22     tx_data_dout,  
23     tx_valid  
24 );
```

```
25  
26 SLAVE SLAVE_instance (  
27     MOSI,  
28     MISO,  
29     SS_n,  
30     clk,  
31     rst_n,  
32     rx_data_din,  
33     rx_valid,  
34     tx_data_dout,  
35     tx_valid  
36 );  
37 endmodule
```

```
1 module WRAPPER_GM (  
2     MOSI,  
3     SS_n,  
4     clk,  
5     rst_n,  
6     MISO  
7 );  
8 input clk, rst_n, MOSI, SS_n;  
9 output MISO;  
10 wire [9:0] rx_data;  
11 wire rx_valid;  
12 wire [7:0] tx_data;  
13 wire tx_valid;  
14 SLAVE_GM SLAVE_GMM (  
15     MOSI,  
16     MISO,  
17     SS_n,  
18     clk,  
19     rst_n,  
20     rx_data,  
21     rx_valid,  
22     tx_data,  
23     tx_valid  
24 );  
25 RAM_GM RAMM_GM (  
26     rx_data,  
27     clk,  
28     rst_n,  
29     rx_valid,  
30     tx_data,  
31     tx_valid
```



# WRAPPER TOP

```
1 import uvm_pkg::*;
2 `include "uvm_macros.svh"
3 import wrapper_test_pkg::*;
4 import config_pkg::*;
5 import config_pkg_spi::*;
6 import config_pkg_ram::*;
7 module top ();
8     bit clk;
9     initial begin
10         clk = 0;
11         forever begin
12             #10 clk = ~clk;
13         end
14     end
15     assign spiif_.MOSI      = DUT.MOSI;
16     assign spiif_.SS_n      = DUT.SS_n;
17     assign spiif_.rst_n     = DUT.rst_n;
18     assign ramif_.din       = DUT.rx_data_din;
19     assign ramif_.rx_valid  = DUT.rx_valid;
20     assign ramif_.rst_n     = DUT.rst_n;
21     assign spiif_.tx_data   = DUT.tx_data_dout;
22     assign spiif_.tx_valid  = DUT.tx_valid;
23     wrapperif wrapperif_ (clk);
24     WRAPPER DUT (
25         wrapperif_.MOSI,
26         wrapperif_.SS_n,
27         clk,
28         wrapperif_.rst_n,
29         wrapperif_.MISO
30     );
```

```
31 WRAPPER_GM DUT_GM (
32     wrapperif_.MOSI,
33     wrapperif_.SS_n,
34     clk,
35     wrapperif_.rst_n,
36     wrapperif_.MISO_exp
37 );
38 spiif spiif_ (clk);
39 SLAVE DUT_SLAVE (
40     spiif_.MOSI,
41     spiif_.MISO,
42     spiif_.SS_n,
43     spiif_.clk,
44     spiif_.rst_n,
45     spiif_.rx_data,
46     spiif_.rx_valid,
47     spiif_.tx_data,
48     spiif_.tx_valid
49 );
50 SLAVE_GM GM_SLAVE (
51     spiif_.MOSI,
52     spiif_.MISO_exp,
53     spiif_.SS_n,
54     spiif_.clk,
55     spiif_.rst_n,
56     spiif_.rx_data_exp,
57     spiif_.rx_valid_exp,
58     spiif_.tx_data,
59     spiif_.tx_valid
60 );
```

```
61 ramif ramif_ (clk);
62 RAM DUT_RAM (
63     ramif_.din,
64     clk,
65     ramif_.rst_n,
66     ramif_.rx_valid,
67     ramif_.dout,
68     ramif_.tx_valid
69 );
70 RAM_GM GM_RAM (
71     ramif_.din,
72     clk,
73     ramif_.rst_n,
74     ramif_.rx_valid,
75     ramif_.dout_exp,
76     ramif_.tx_valid_exp
77 );
78 initial begin
79     uvm_config_db#(virtual spiif)::set(null, "uvm_test_top", "SPI", spiif_);
80     uvm_config_db#(virtual ramif)::set(null, "uvm_test_top", "RAM", ramif_);
81     uvm_config_db#(virtual wrapperif)::set(null, "uvm_test_top", "wrapper", wrapperif_);
82     run_test("wrapper_test");
83 end
84
85 bind RAM ram_SVA sva_instance (
86     ramif_.din,
87     clk,
88     ramif_.rst_n,
89     ramif_.rx_valid,
90     ramif_.dout,
91     ramif_.tx_valid
92 );
93 bind WRAPPER wrapper_SVA sva_instance_wrapper (
94     wrapperif_.clk,
95     wrapperif_.rst_n,
96     wrapperif_.MOSI,
97     wrapperif_.MISO,
98     wrapperif_.SS_n
99 );
100
101 endmodule
```

# WRAPPER\_IF|CONFIG

```
1  ✓ interface wrapperif (  
2      |   clk  
3  ✓ );  
4      input clk;  
5      logic MOSI, SS_n, rst_n;  
6      logic MISO, MISO_exp;  
7  endinterface  
8
```

```
1  package config_pkg_wrapper;  
2      import uvm_pkg::*;  
3      `include "uvm_macros.svh"  
4      class wrapper_config_obj extends uvm_object;  
5          |   `uvm_object_utils(wrapper_config_obj);  
6          virtual wrapperif wrapper_config_vif;  
7          uvm_active_passive_enum is_active;  
8          function new(string name = "wrapper_config");  
9              |   super.new(name);  
10             endfunction //new()  
11         endclass  
12     endpackage  
13
```

# WRAPPER\_TEST

```
1 package wrapper_test_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import spi_env_pkg::*;
5 import ram_env_pkg::*;
6 import wrapper_env_pkg::*;
7 import config_pkg_ram::*;
8 import seq_rand_pkg::*;
9 import config_pkg_spi::*;
10 import config_pkg_wrapper::*;
11 import seq_wr_only_pkg::*;
12 import seq_rd_only_pkg::*;
13 import seq_rst_pkg::*;
14 import seq_wr_rd_only_pkg::*;
15 class wrapper_test extends uvm_test;
16     `uvm_component_utils(wrapper_test)
17     spi_env spi_env_comp;
18     ram_env ram_env_comp;
19     wrapper_env wrapper_env_comp;
20     spi_config_obj spi_config_obj_test;
21     ram_config_obj ram_config_obj_test;
22     wrapper_config_obj wrapper_config_obj_test;
23     //seq_rand randd;
24     seq_rst seq_rst_;
25     seq_wr_only seq_wr_only_;
26     seq_rd_only seq_rd_only_;
27     seq_wr_rd_only seq_wr_rd_only_;
28     function new(string name = "wrapper_test", uvm_component parent = null);
29         super.new(name, parent);
30     endfunction
```

```
31 function void build_phase(uvm_phase phase);
32     super.build_phase(phase);
33     spi_env_comp = spi_env::type_id::create("spi_env_comp", this);
34     ram_env_comp = ram_env::type_id::create("ram_env_comp", this);
35     wrapper_env_comp = wrapper_env::type_id::create("wrapper_env_comp", this);
36     spi_config_obj_test = spi_config_obj::type_id::create("spi_config_comp", this);
37     ram_config_obj_test = ram_config_obj::type_id::create("ram_config_comp", this);
38     wrapper_config_obj_test = wrapper_config_obj::type_id::create("wrapper_config_comp", this);
39     spi_config_obj_test.is_active = UVM_PASSIVE;
40     ram_config_obj_test.is_active = UVM_PASSIVE;
41     wrapper_config_obj_test.is_active = UVM_ACTIVE;
42     //randd = seq_rand::type_id::create("randd");
43     seq_rst_ = seq_rst::type_id::create("rst");
44     seq_wr_only_ = seq_wr_only::type_id::create("wr");
45     seq_rd_only_ = seq_rd_only::type_id::create("rd");
46     seq_wr_rd_only_ = seq_wr_rd_only::type_id::create("seq_wr_rd");
47     if (~uvm_config_db#(virtual spiif)::get(this, "", "SPI", spi_config_obj_test.spi_config_vif))
48         `uvm_fatal("build_phase", "Test -Unable to get into virtual interface_spi");
49     if (~uvm_config_db#(virtual ramif)::get(this, "", "RAM", ram_config_obj_test.ram_config_vif))
50         `uvm_fatal("build_phase", "Test -Unable to get into virtual interface_ram");
51     if (~uvm_config_db#(virtual wrapperif)::get(
52         this, "", "wrapper", wrapper_config_obj_test.wrapper_config_vif
53     ))
54         `uvm_fatal("build_phase", "Test -Unable to get into virtual interface_wrapper");
55     uvm_config_db#(spi_config_obj)::set(this, "*", "SPI", spi_config_obj_test);
56     uvm_config_db#(ram_config_obj)::set(this, "*", "RAM", ram_config_obj_test);
57     uvm_config_db#(wrapper_config_obj)::set(this, "*", "wrapper", wrapper_config_obj_test);
58 endfunction
```

# WRAPPER\_TEST

```
59 task run_phase(uvm_phase phase);
60     super.run_phase(phase);
61     phase.raise_objection(this);
62     `uvm_info("run_phase", "rst_begin", UVM_LOW);
63     seq_rst_.start(wrapper_env_comp.agent.sqr);
64     `uvm_info("run_phase", "rst_end", UVM_LOW);
65     `uvm_info("run_phase", "wr_begin", UVM_LOW);
66     seq_wr_only_.start(wrapper_env_comp.agent.sqr);
67     `uvm_info("run_phase", "wr_end", UVM_LOW);
68     `uvm_info("run_phase", "rd_begin", UVM_LOW);
69     seq_rd_only_.start(wrapper_env_comp.agent.sqr);
70     `uvm_info("run_phase", "rd_end", UVM_LOW);
71     `uvm_info("run_phase", "wr_rd_begin", UVM_LOW);
72     seq_wr_rd_only_.start(wrapper_env_comp.agent.sqr);
73     `uvm_info("run_phase", "wr_rd_end", UVM_LOW);
74     /* `uvm_info("run_phase", "randd_begin", UVM_LOW);
75     randd.start(wrapper_env_comp.agent.sqr);
76     `uvm_info("run_phase", "randd_end", UVM_LOW);*/
77     phase.drop_objection(this);
78 endtask
79 endclass
80 endpackage
81
```



# WRAPPER\_ENV

```
1 package wrapper_env_pkg;
2   import uvm_pkg::*;
3   import wrapper_driver_pkg::*;
4   import config_pkg_wrapper::*;
5   import sequencer_pkg::*;
6   import wrapper_monitor_pkg::*;
7   import wrapper_seq_item_pkg::*;
8   import wrapper_agent_pkg::*;
9   import sb_pkg::*;
10  //import sb_pkg::*;
11  import cvg_pkg::*;
12  `include "uvm_macros.svh"
13  class wrapper_env extends uvm_env;
14    `uvm_component_utils(wrapper_env)
15    wrapper_agent agent;
16    wrapper_cover_gp cgp;
17    wrapper_seq_item seq_item;
18    wrapper_sb sb;
19    function new(string name = "wrapper_env", uvm_component parent = null);
20      super.new(name, parent);
21    endfunction //new()
22    function void build_phase(uvm_phase phase);
23      super.build_phase(phase);
24      agent = wrapper_agent::type_id::create("agent", this);
25      sb = wrapper_sb::type_id::create("sb", this);
26      cgp = wrapper_cover_gp::type_id::create("cgp", this);
27    endfunction
28    function void connect_phase(uvm_phase phase);
29      super.connect_phase(phase);
30      agent.agt_ap.connect(sb.sb_export);
31      agent.agt_ap.connect(cgp.cgp_export);
32    endfunction
33  endclass
34 endpackage
```



# WRAPPER\_agent

```
1 package wrapper_agent_pkg;
2   import uvm_pkg::*;
3   import wrapper_driver_pkg::*;
4   import config_pkg_wrapper::*;
5   import wrapper_sequencer_pkg::*;
6   import wrapper_monitor_pkg::*;
7   import wrapper_seq_item_pkg::*;
8   `include "uvm_macros.svh"
9   class wrapper_agent extends uvm_agent;
10     `uvm_component_utils(wrapper_agent)
11     wrapper_sequencer sqr;
12     wrapper_driver drv;
13     wrapper_config_obj cnfg;
14     wrapper_monitor mon;
15     uvm_analysis_port #(wrapper_seq_item) agt_ap;
16     function new(string name = "wrapper_agent", uvm_component parent = null);
17       super.new(name, parent);
18     endfunction
19     function void build_phase(uvm_phase phase);
20       super.build_phase(phase);
21       agt_ap = new("agt_ap", this);
22       if (~uvm_config_db#(wrapper_config_obj)::get(this, "", "wrapper", cnfg)) begin
23         `uvm_fatal("build_phase", "unable to conf");
24       end
25       if (cnfg.is_active == UVM_ACTIVE) begin
26         sqr = wrapper_sequencer::type_id::create("sqr", this);
27         drv = wrapper_driver::type_id::create("drv", this);
28       end
29       mon = wrapper_monitor::type_id::create("mon", this);
30     endfunction
31     function void connect_phase(uvm_phase phase);
32       super.connect_phase(phase);
33       mon.wrapper_monitor_if = cnfg.wrapper_config_vif;
34       if (cnfg.is_active == UVM_ACTIVE) begin
35         drv.seq_item_port.connect(sqr.seq_item_export);
36         drv.wrapper_driver_vif = cnfg.wrapper_config_vif;
37       end
38       mon.mon_ap.connect(agt_ap);
39     endfunction
40   endclass
41 endpackage
42
```

# SPI|RAM\_agent

```
9 class spi_agent extends uvm_agent;
10   `uvm_component_utils(spi_agent)
11   spi_sequencer sqr;
12   spi_driver drv;
13   spi_config_obj cnfg;
14   spi_monitor mon;
15   uvm_analysis_port #(spi_seq_item) agt_ap;
16   function new(string name = "spi_agent", uvm_component parent = null);
17     super.new(name, parent);
18   endfunction
19   function void build_phase(uvm_phase phase);
20     super.build_phase(phase);
21     agt_ap = new("agt_ap", this);
22     if (~uvm_config_db#(spi_config_obj)::get(this, "", "SPI", cnfg)) begin
23       `uvm_fatal("build_phase", "unable to conf");
24     end
25     if (cnfg.is_active == UVM_ACTIVE) begin
26       sqr = spi_sequencer::type_id::create("sqr", this);
27       drv = spi_driver::type_id::create("drv", this);
28     end
29     mon = spi_monitor::type_id::create("mon", this);
30   endfunction
31   function void connect_phase(uvm_phase phase);
32     super.connect_phase(phase);
33     mon.spi_monitor_if = cnfg.spi_config_vif;
34     if (cnfg.is_active == UVM_ACTIVE) begin
35       drv.spi_driver_vif = cnfg.spi_config_vif;
36       drv.seq_item_port.connect(sqr.seq_item_export);
37     end
38     mon.mon_ap.connect(agt_ap);
39   endfunction
40 endclass
41 endpackage
```

```
9 class ram_agent extends uvm_agent;
10   `uvm_component_utils(ram_agent)
11   ram_sequencer sqr;
12   ram_driver drv;
13   ram_config_obj cnfg;
14   ram_monitor mon;
15   uvm_analysis_port #(ram_seq_item) agt_ap;
16   function new(string name = "ram_agent", uvm_component parent = null);
17     super.new(name, parent);
18   endfunction
19   function void build_phase(uvm_phase phase);
20     super.build_phase(phase);
21     agt_ap = new("agt_ap", this);
22     if (~uvm_config_db#(ram_config_obj)::get(this, "", "RAM", cnfg)) begin
23       `uvm_fatal("build_phase", "unable to conf");
24     end
25     if (cnfg.is_active == UVM_ACTIVE) begin
26       sqr = ram_sequencer::type_id::create("sqr", this);
27       drv = ram_driver::type_id::create("drv", this);
28     end
29     mon = ram_monitor::type_id::create("mon", this);
30   endfunction
31   function void connect_phase(uvm_phase phase);
32     super.connect_phase(phase);
33     mon.ram_monitor_if = cnfg.ram_config_vif;
34     if (cnfg.is_active == UVM_ACTIVE) begin
35       drv.seq_item_port.connect(sqr.seq_item_export);
36       drv.ram_driver_vif = cnfg.ram_config_vif;
37     end
38     mon.mon_ap.connect(agt_ap);
39   endfunction
40 endclass
41 endpackage
```

# WRAPPER\_MONITOR|DRIVER

```
1 package wrapper_monitor_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import wrapper_seq_item_pkg::*;
5 class wrapper_monitor extends uvm_monitor;
6     `uvm_component_utils(wrapper_monitor);
7     virtual wrapperif wrapper_monitor_if;
8     wrapper_seq_item seq_item;
9     uvm_analysis_port #(wrapper_seq_item) mon_ap;
10    function new(string name = "wrapper_monitor", uvm_component parent = null);
11        super.new(name, parent);
12    endfunction
13    function void build_phase(uvm_phase phase);
14        super.build_phase(phase);
15        mon_ap = new("mon_ap", this);
16    endfunction
17    task run_phase(uvm_phase phase);
18        super.run_phase(phase);
19        forever begin
20            seq_item = wrapper_seq_item::type_id::create("seq_item_wrapper");
21            //we must sample the outputs of the DUT
22            @(negedge wrapper_monitor_if.clk);
23            seq_item.MOSI = wrapper_monitor_if.MOSI;
24            seq_item.rst_n = wrapper_monitor_if.rst_n;
25            seq_item.SS_n = wrapper_monitor_if.SS_n;
26            seq_item.MISO = wrapper_monitor_if.MISO;
27            mon_ap.write(seq_item);
28            //also at the same time sample the outputs of the GOLDEN MODEL
29            seq_item.MISO_exp = wrapper_monitor_if.MISO_exp;
30            `uvm_info("run_phase", seq_item.convert2string(), UVM_HIGH);
31        end
32    endtask
33 endclass
34 endpackage
35
```

```
1 package wrapper_driver_pkg;
2 import uvm_pkg::*;
3 import config_pkg_wrapper::*;
4 import wrapper_seq_item_pkg::*;
5 `include "uvm_macros.svh"
6 class wrapper_driver extends uvm_driver #(wrapper_seq_item);
7     `uvm_component_utils(wrapper_driver)
8     virtual wrapperif wrapper_driver_vif;
9     wrapper_config_obj wrapper_config_obj_driver;
10    wrapper_seq_item seq_item;
11    function new(string name = "wrapper_driver", uvm_component parent = null);
12        super.new(name, parent);
13    endfunction //new()
14    task run_phase(uvm_phase phase);
15        super.run_phase(phase);
16        forever begin
17            seq_item = wrapper_seq_item::type_id::create("seq_item_driver");
18            seq_item_port.get_next_item(seq_item);
19            wrapper_driver_vif.MOSI = seq_item.MOSI;
20            wrapper_driver_vif.rst_n = seq_item.rst_n;
21            wrapper_driver_vif.SS_n = seq_item.SS_n;
22            @(negedge wrapper_driver_vif.clk);
23            seq_item_port.item_done();
24        end
25    endtask
26 endclass
27 endpackage
28
```



# WRAPPER\_SEQ\_ITEM

```
1 package wrapper_seq_item_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 class wrapper_seq_item extends uvm_sequence_item;
5     `uvm_object_utils(wrapper_seq_item)
6     logic MOSI;
7     rand logic SS_n;
8     rand logic rst_n;
9     logic MISO, MISO_exp;
10    logic [7:0] dout_exp;
11    rand bit [7:0] array_;
12    bit [10:0] array_final, prev_array;
13    rand bit [2:0] state;
14    bit is_read;
15    bit wr_add;
16    bit wr_data;
17    bit rd_add, wr_data_done, rd_add_, rd_data;
18    int counter;
19    bit communication;
20    int index;
21    constraint write_only_seq {state inside {3'b000, 3'b001};}
22    constraint read_only_seq {
23        if (rd_add_)
24            state inside {3'b111};
25        else
26            if (rd_data)
27                state inside {3'b110};
28            else
29                state inside {3'b111, 3'b110};
30    }
31    constraint wr_rd_seq {
32        if (wr_data) state inside {3'b000, 3'b001};
33        if (wr_data_done)
34            state dist {
35                3'b000 := 40,
36                3'b110 := 60
37            };
38        if (rd_add_) state inside {3'b111};
39        if (rd_data)
40            state dist {
41                3'b000 := 60,
42                3'b110 := 40
43            };
44    }
45    function new(string name = "wrapper_seq_item");
46        super.new(name);
47    endfunction
48    constraint rst_n_cons {
49        rst_n dist {
50            1 := 98,
51            0 := 2
52        };
53    }
54    constraint SS_n_high {SS_n == (!communication);}
55    function write_only_seq {state inside {3'b000, 3'b001};}
```

# WRAPPER\_SEQ\_ITEM

```
55 function void post_randomize();
56 if (SS_n) begin
57     array_final = {state, array_};
58     if (state == 3'b111) begin
59         is_read = 1;
60     end else is_read = 0;
61 end
62 //STATE SEQ_HANDLE
63 if (state == 3'b000) begin
64     wr_data = 1;
65     wr_data_done = 0;
66 end else if (state == 3'b001) begin
67     wr_data = 0;
68     wr_data_done = 1;
69 end else begin
70     wr_data = 0;
71     wr_data_done = 0;
72 end //wr_handle
73 if (state == 3'b110) begin
74     rd_add_ = 1;
75     rd_data = 0;
76 end else if (state == 3'b111) begin
77     rd_data = 1;
78     rd_add_ = 0;
79 end else begin
80     rd_add_ = 0;
81     rd_data = 0;
82 end
83 //SS_n handle
84 if (is_read) begin
85     if (counter == 23) begin
86         communication = 0;
87         counter = 0;
88     end else begin
```

```
88     end else begin
89         communication = 1;
90         counter++;
91     end
92 end else begin
93     if (counter == 13) begin
94         communication = 0;
95         counter = 0;
96     end else begin
97         communication = 1;
98         counter++;
99     end
100 end
101 //MOSI HANDLE
102 if (~SS_n) begin
103     if (index >= 0) begin
104         MOSI = array_final[index];
105         index = index - 1;
106     end else begin
107         MOSI = 0;
108         index = 10;
109     end
110 end else begin
111     MOSI = 0;
112     index = 10;
113 end
114 endfunction
```

```
115 function string convert2string();
116 return $sformatf(
117     "%s MOSI=%0d, rst_n=0b%0b,SS_n=0b%0b,MISO=0b%0b",
118     super.convert2string(),
119     MOSI,
120     rst_n,
121     SS_n,
122     MISO,
123 );
124 endfunction
125 function string convert2string_stimulus();
126 return $sformatf("MOSI=%0d, rst_n=%0b,SS_n=0b%0b, MISO=0b%0b", MOSI, rst_n, SS_n, MISO,);
127 endfunction
128 endclass
129 endpackage
130
```



# WRAPPER\_RST\_SEQ

```
1  package seq_rst_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4      import wrapper_seq_item_pkg::*;
5      class seq_rst extends uvm_sequence #(wrapper_seq_item);
6          `uvm_object_utils(seq_rst);
7          wrapper_seq_item seq_item;
8          function new(string name = "seq_rst");
9              super.new(name);
10         endfunction
11         task body;
12             seq_item = wrapper_seq_item::type_id::create("seq_item_rst");
13             repeat (100) begin
14                 start_item(seq_item);
15                 seq_item.rst_n = 0;
16                 finish_item(seq_item);
17             end
18         endtask
19     endclass //seq_rst
20 endpackage
```

# WRAPPER\_WR\_ONLY\_SEQ

```
1  package seq_wr_only_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4      import wrapper_seq_item_pkg::*;
5  class seq_wr_only extends uvm_sequence #(wrapper_seq_item);
6      `uvm_object_utils(seq_wr_only);
7      wrapper_seq_item seq_item;
8  function new(string name = "seq_wr_only");
9      super.new(name);
10 endfunction
11 task body;
12     seq_item = wrapper_seq_item::type_id::create("seq_item_wr_only");
13     repeat (10000) begin
14         start_item(seq_item);
15         if (seq_item.counter == 0) begin
16             seq_item.write_only_seq.constraint_mode(1);
17             seq_item.read_only_seq.constraint_mode(0);
18             seq_item.wr_rd_seq.constraint_mode(0);
19         end else begin
20             seq_item.write_only_seq.constraint_mode(0);
21             seq_item.read_only_seq.constraint_mode(0);
22             seq_item.wr_rd_seq.constraint_mode(0);
23         end
24         assert (seq_item.randomize());
25         finish_item(seq_item);
26     end
27 endtask
28 endclass //seq_wr_only
29 endpackage
30
```

# WRAPPER\_RD\_ONLY\_SEQ

```
1  package seq_rd_only_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4      import wrapper_seq_item_pkg::*;
5  class seq_rd_only extends uvm_sequence #(wrapper_seq_item);
6      `uvm_object_utils(seq_rd_only);
7      wrapper_seq_item seq_item;
8  function new(string name = "seq_rd_only");
9      super.new(name);
10 endfunction
11 task body;
12     seq_item = wrapper_seq_item::type_id::create("seq_item_rd_only");
13     repeat (10000) begin
14         start_item(seq_item);
15         if (seq_item.counter == 0) begin
16             seq_item.write_only_seq.constraint_mode(0);
17             seq_item.read_only_seq.constraint_mode(1);
18             seq_item.wr_rd_seq.constraint_mode(0);
19         end else begin
20             seq_item.write_only_seq.constraint_mode(0);
21             seq_item.read_only_seq.constraint_mode(0);
22             seq_item.wr_rd_seq.constraint_mode(0);
23         end
24         assert (seq_item.randomize());
25         finish_item(seq_item);
26     end
27 endtask
28 endclass //seq_rd_only
29 endpackage
```

# WRAPPER\_WR\_RD\_SEQ

```
1 package seq_wr_rd_only_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import wrapper_seq_item_pkg::*;
5 class seq_wr_rd_only extends uvm_sequence #(wrapper_seq_item);
6     `uvm_object_utils(seq_wr_rd_only);
7     wrapper_seq_item seq_item;
8     function new(string name = "seq_wr_rd_only");
9         super.new(name);
10    endfunction
11    task body;
12        seq_item = wrapper_seq_item::type_id::create("seq_item_wr_rd_only");
13        repeat (10000) begin
14            start_item(seq_item);
15            if (seq_item.counter == 0) begin
16                seq_item.write_only_seq.constraint_mode(0);
17                seq_item.read_only_seq.constraint_mode(0);
18                seq_item.wr_rd_seq.constraint_mode(1);
19            end else begin
20                seq_item.write_only_seq.constraint_mode(0);
21                seq_item.read_only_seq.constraint_mode(0);
22                seq_item.wr_rd_seq.constraint_mode(0);
23            end
24            assert (seq_item.randomize());
25            finish_item(seq_item);
26        end
27    endtask
28 endclass
29 endpackage
```

# WRAPPER\_SQR

```
1  package wrapper_sequencer_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4      import wrapper_seq_item_pkg::*;
5      import seq_rand_pkg::*;
6      class wrapper_sequencer extends uvm_sequencer #(wrapper_seq_item);
7          `uvm_component_utils(wrapper_sequencer)
8          function new(string name = "wrapper_sequencer", uvm_component parent = null);
9              super.new(name, parent);
10         endfunction
11     endclass
12 endpackage
13
```



# WRAPPER\_COVERGROUPS

```
1  package cvg_pkg;
2      import config_pkg_wrapper::*;
3      import wrapper_seq_item_pkg::*;
4      import uvm_pkg::*;
5      import wrapper_agent_pkg::*;
6      `include "uvm_macros.svh"
7      class wrapper_cover_gp extends uvm_component;
8          `uvm_component_utils(wrapper_cover_gp);
9          uvm_analysis_export #(wrapper_seq_item) cgp_export;
10         uvm_tlm_analysis_fifo #(wrapper_seq_item) cgp_fifo;
11         wrapper_seq_item seq_item;
12         covergroup cvr_gp;
13             ///MORE COVERAGE IF NEEDED
14         endgroup : cvr_gp
15         function new(string name = "cgp", uvm_component parent = null);
16             super.new(name, parent);
17             cvr_gp = new();
18         endfunction
19         function void build_phase(uvm_phase phase);
20             super.build_phase(phase);
21             cgp_export = new("cgp_export", this);
22             cgp_fifo    = new("cgp_fifo", this);
23         endfunction
24         function void connect_phase(uvm_phase phase);
25             super.connect_phase(phase);
26             cgp_export.connect(cgp_fifo.analysis_export);
27         endfunction
28         task run_phase(uvm_phase phase);
29             super.run_phase(phase);
30             forever begin
31                 cgp_fifo.get(seq_item);
32                 cvr_gp.sample();
33             end
34         endtask
35     endclass
36
37 endpackage
```

# WRAPPER\_SVA

```
1  module wrapper_SVA (  
2      clk,  
3      rst_n,  
4      MOSI,  
5      MISO,  
6      SS_n  
7  );  
8      input MOSI, SS_n, clk, rst_n;  
9      input MISO;  
10  
11     property rst_n_check;  
12         @(posedge clk) !rst_n | => !MISO;  
13     endproperty  
14  
15     assert property (rst_n_check)  
16     else $error("MISO should be low when reset is asserted");  
17  
18     cover property (rst_n_check);  
19  
20     sequence READ_DATA_SEQUENCE;  
21         $fell(  
22             SS_n  
23         ) ##1 (MOSI [-> 3] ##0 1'b1);  
24     endsequence  
25  
26     property MISO_STABLE_NOT_READ;  
27         @(posedge clk) disable iff (!rst_n) $fell(  
28             SS_n  
29         ) | => (not READ_DATA_SEQUENCE ##1  
30             ($stable(  
31                 MISO  
32             ) throughout (!SS_n)));  
33     endproperty  
34  
35     assert property (MISO_STABLE_NOT_READ)  
36     else $error("MISO changed during SS_n low in non-read sequence");  
37
```

# WRAPPER\_SVA

| ASSERTION_NAME              | DESCRIPTION                                                                                                                  | ASSERTION                                                                                                               |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>rst_n_check</b>          | When reset is asserted, MISO should be low on the next clock cycle                                                           | <code>@(posedge clk) !rst_n   =&gt; !MISO</code>                                                                        |
| <b>MISO_STABLE_NOT_READ</b> | During non-read sequences (when SS_n is low but not in read data mode), MISO should remain stable throughout the transaction | <code>disable iff (!rst_n) \$fell(SS_n)   =&gt; (not READ_DATA_SEQUENCE ##1 (\$stable(MISO) throughout (!SS_n)))</code> |

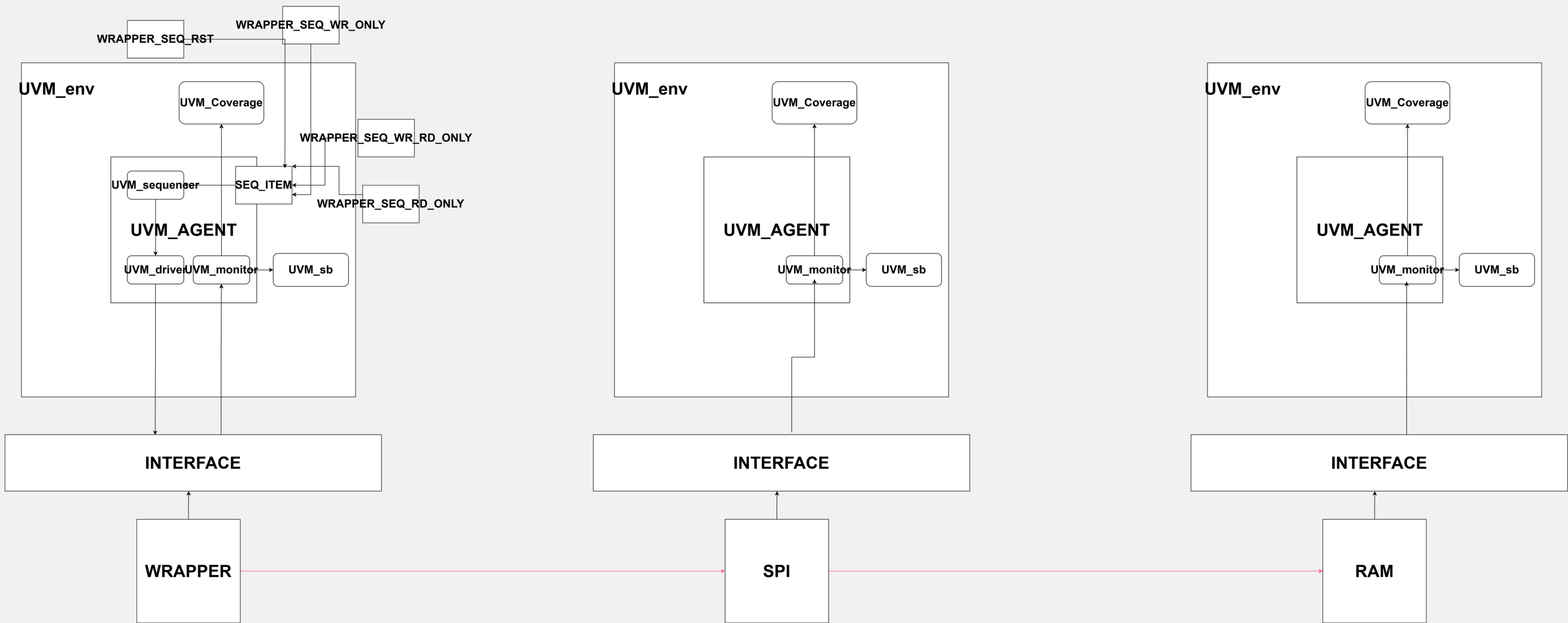
# WRAPPER\_PLAN

| LABEL           | DESIGN REQUIREMENT DESCRIPTION                                                                                                                                                                                                                                                                   | STIMULUS GENERATION                                                                                    | FUNCTIONAL COVERAGE                                                 | FUNCTIONALITY CHECK                                       |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------|
| WRAPPER_RST_SEQ | when rst_n is 0 outputs must be equal zero                                                                                                                                                                                                                                                       | a seq that make the rst_n=0                                                                            | -                                                                   | rst assertion is made also by comparing to a golden model |
| WR_ONLY_SEQ     | wr_only is a seq inside 000 001 and make the ram either write address or data                                                                                                                                                                                                                    | by turning on and off a certain constraint and that constraint is a write_only_seq it includes 000 001 | bins transitions are made to insure that wr_address or data happens | comparing with golden model and by assertions             |
| RD_ONLY_SEQ     | every Read Address operation shall always be followed by Read Data. After a Read Data operation shall always be followed by Read Address.                                                                                                                                                        | by turning on and off constraints and only constraint of RD_ONLY_SEQ is on                             | transition bins are made to insure that the wanted seq occurred     | comparing with golden model and by assertions             |
| WR_RD_SEQ       | A Write Address is always followed by either another Write Address or Write Data. After Write Data, the next operation is 60% Read Address and 40% Write Address. A Read Address is always followed by Read Data. After Read Data, the next operation is 60% Write Address and 40% Read Address. | turning off all constraints except WR_RD_ONLY_SEQ                                                      | transition bins are made to insure that the wanted seq occurred     | comparing with golden model and by assertions             |

# WRAPPER\_PLAN

|           |                                                                             |                                                                                                   |                                                                                                                                                                    |                                                                                                              |
|-----------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| WRAPPER_1 | When SS_n is 0 and MOSI becomes 0 0 0 a write address operation must occure | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when wr_data seq happens a bin is made to insure that a wr_add seq is made also at the same time a SS_n is made to ensure that SS_n is low for 13 clk cycle        | VALID_wr_add_seq assertion is made to insure that it works correctly<br>also comparing to a golden model     |
| WRAPPER_2 | When SS_n is 0 and MOSI becomes 0 0 1 a write data operation must occure    | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when wr_data seq happens a bin is made to insure that a wr_data seq is made also at the same time a SS_n is made to ensure that SS_n is low for 13 clk cycle       | a VALID_wr_data _seq assertion is made to insure that it works correctly<br>also comparing to a golden model |
| WRAPPER_3 | When SS_n is 0 and MOSI becomes 1 1 0 a read address operation must occure  | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when rd_address seq happens a bin is made to insure that a rd_address seq is made also at the same time a SS_n is made to ensure that SS_n is low for 13 clk cycle | VALID_rd_address_seq assertion is made to insure that it works correctly<br>also comparing to a golden model |
| WRAPPER_4 | When SS_n is 0 and MOSI becomes 1 1 1 a read data operation must occure     | by randomizing with certain constraints<br>as only write address , read address , wr_data,rd_data | when rd_data seq happens a bin is made to insure that a rd_data seq is made also at the same time a SS_n is made to ensure that SS_n is low for 23 clk cycle       | VALID_rd_data_seq assertion is made to insure that it works correctly<br>also comparing to a golden model    |





# WRAPPER\_ASSERTIONS

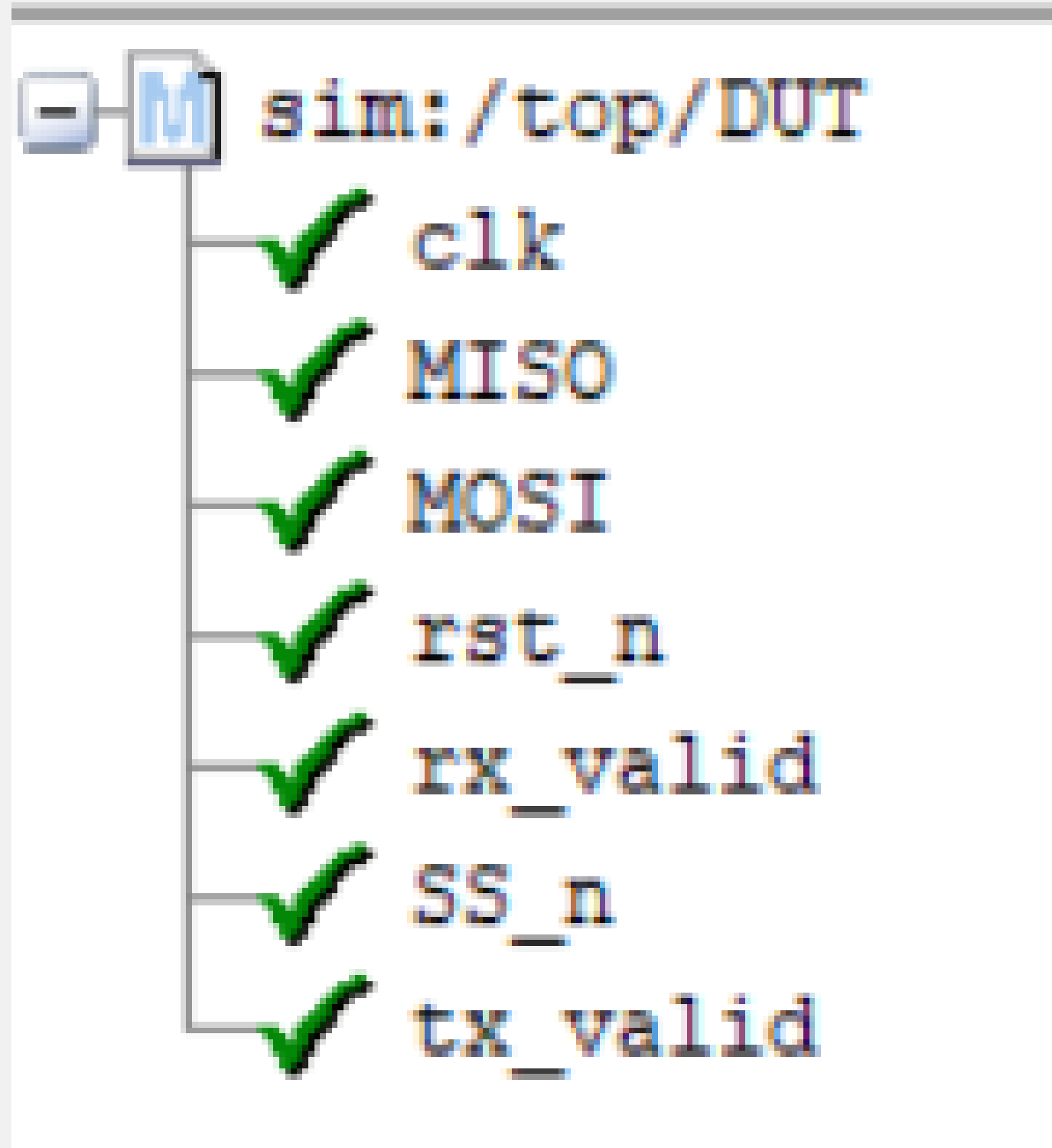
|                                                                            |            |     |    |   |       |    |        |       |           |       |                         |                                          |   |
|----------------------------------------------------------------------------|------------|-----|----|---|-------|----|--------|-------|-----------|-------|-------------------------|------------------------------------------|---|
| ▲ /uvm_pkg::uvm_reg_map::do_write/#ublk#215181159#1731/immed__1735         | Immediate  | SVA | on | 0 | 0     | 0  | -      | -     | -         | off   | assert (\$cast(seq,o))  | ✗                                        |   |
| ▲ /uvm_pkg::uvm_reg_map::do_read/#ublk#215181159#1771/immed__1775          | Immediate  | SVA | on | 0 | 0     | 0  | -      | -     | -         | off   | assert (\$cast(seq,o))  | ✗                                        |   |
| ▲ /seq_rand_pkg::seq_rand::body/#ublk#30806423#13/immed__15                | Immediate  | SVA | on | 0 | 0     | 0  | -      | -     | -         | off   | assert (randomize(...)) | ✗                                        |   |
| ▲ /seq_wr_rd_only_pkg::seq_wr_rd_only::body/#ublk#114004951#13/immed__2... | Immediate  | SVA | on | 0 | 10000 | 0  | -      | -     | -         | off   | assert (randomize(...)) | ✓                                        |   |
| ▲ /seq_rd_only_pkg::seq_rd_only::body/#ublk#72850615#13/immed__24          | Immediate  | SVA | on | 0 | 10000 | 0  | -      | -     | -         | off   | assert (randomize(...)) | ✓                                        |   |
| ▲ /seq_wr_only_pkg::seq_wr_only::body/#ublk#72449207#13/immed__24          | Immediate  | SVA | on | 0 | 10000 | 0  | -      | -     | -         | off   | assert (randomize(...)) | ✓                                        |   |
| +▲ /top/DUT/RAM_instance/sva_instance/assert_rst_n_prop                    | Concurrent | SVA | on | 0 | 682   | 0  | 0B     | 160B  | 30 ns     | 682   | off                     | assert( @(posedge clk) (~rst_n) =...     | ✓ |
| +▲ /top/DUT/RAM_instance/sva_instance/assert_assertion_2_tx_valid_low      | Concurrent | SVA | on | 0 | 22804 | 1  | 80B    | 160B  | 2030 ns   | 23251 | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/RAM_instance/sva_instance/assert_tx_valid_high                 | Concurrent | SVA | on | 0 | 1133  | 0  | 0B     | 960B  | 229470 ns | 6167  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/RAM_instance/sva_instance/assert_rd_add_data                   | Concurrent | SVA | on | 0 | 3297  | 0  | 0B     | 10.7K | 219390 ns | 10671 | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/RAM_instance/sva_instance/assert_rd_data_add                   | Concurrent | SVA | on | 0 | 4125  | 21 | 1.7K.. | 3.4K  | 207570 ns | 10292 | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_rst_n_prop                                | Concurrent | SVA | on | 0 | 682   | 0  | 0B     | 160B  | 30 ns     | 682   | off                     | assert( @(posedge clk) (~rst_n) =...     | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_VALID_wr_add_seq                          | Concurrent | SVA | on | 0 | 216   | 1  | 80B    | 80B   | 2030 ns   | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_VALID_wr_data_seq                         | Concurrent | SVA | on | 0 | 345   | 1  | 80B    | 160B  | 202030 ns | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_VALID_rd_add_seq                          | Concurrent | SVA | on | 0 | 150   | 1  | 80B    | 160B  | 402030 ns | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_VALID_rd_data_seq                         | Concurrent | SVA | on | 0 | 120   | 1  | 80B    | 160B  | 409070 ns | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_first                                     | Concurrent | SVA | on | 0 | 2329  | 0  | 0B     | 80B   | 2030 ns   | 2329  | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_sec                                       | Concurrent | SVA | on | 0 | 2285  | 0  | 0B     | 80B   | 2050 ns   | 2285  | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_third                                     | Concurrent | SVA | on | 0 | 992   | 0  | 0B     | 80B   | 2290 ns   | 992   | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_forth                                     | Concurrent | SVA | on | 0 | 386   | 0  | 0B     | 80B   | 2850 ns   | 386   | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT/SLAVE_instance/ASSRT_fifth                                     | Concurrent | SVA | on | 0 | 388   | 0  | 0B     | 80B   | 203050 ns | 388   | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT/sva_instance_wrapper/assert__rst_n_check                       | Concurrent | SVA | on | 0 | 682   | 0  | 0B     | 160B  | 30 ns     | 682   | off                     | assert( @(posedge clk) (~rst_n) =...     | ✓ |
| +▲ /top/DUT/sva_instance_wrapper/assert__MISO_STABLE_NOT_READ              | Concurrent | SVA | on | 0 | 1804  | 1  | 80B    | 160B  | 2050 ns   | 3639  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_rst_n_prop                                         | Concurrent | SVA | on | 0 | 682   | 0  | 0B     | 160B  | 30 ns     | 682   | off                     | assert( @(posedge clk) (~rst_n) =...     | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_VALID_wr_add_seq                                   | Concurrent | SVA | on | 0 | 216   | 1  | 80B    | 80B   | 2030 ns   | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_VALID_wr_data_seq                                  | Concurrent | SVA | on | 0 | 345   | 1  | 80B    | 160B  | 202030 ns | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_VALID_rd_add_seq                                   | Concurrent | SVA | on | 0 | 150   | 1  | 80B    | 160B  | 402030 ns | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_VALID_rd_data_seq                                  | Concurrent | SVA | on | 0 | 120   | 1  | 80B    | 160B  | 409070 ns | 1835  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_first                                              | Concurrent | SVA | on | 0 | 2329  | 0  | 0B     | 80B   | 2030 ns   | 2329  | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_sec                                                | Concurrent | SVA | on | 0 | 2285  | 0  | 0B     | 80B   | 2050 ns   | 2285  | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_third                                              | Concurrent | SVA | on | 0 | 992   | 0  | 0B     | 80B   | 2290 ns   | 992   | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_forth                                              | Concurrent | SVA | on | 0 | 386   | 0  | 0B     | 80B   | 2850 ns   | 386   | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT_SLAVE/ASSRT_fifth                                              | Concurrent | SVA | on | 0 | 388   | 0  | 0B     | 80B   | 203050 ns | 388   | off                     | assert( disable iff (~rst_n) ( @(pos...  | ✓ |
| +▲ /top/DUT_RAM/sva_instance/assert_rst_n_prop                             | Concurrent | SVA | on | 0 | 682   | 0  | 0B     | 160B  | 30 ns     | 682   | off                     | assert( @(posedge clk) (~rst_n) =...     | ✓ |
| +▲ /top/DUT_RAM/sva_instance/assert_assertion_2_tx_valid_low               | Concurrent | SVA | on | 0 | 22804 | 1  | 80B    | 160B  | 2030 ns   | 23251 | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_RAM/sva_instance/assert_tx_valid_high                          | Concurrent | SVA | on | 0 | 1133  | 0  | 0B     | 960B  | 229470 ns | 6167  | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_RAM/sva_instance/assert_rd_add_data                            | Concurrent | SVA | on | 0 | 3297  | 0  | 0B     | 10.7K | 219390 ns | 10671 | off                     | assert( @(posedge clk) disable iff (...) | ✓ |
| +▲ /top/DUT_RAM/sva_instance/assert_rd_data_add                            | Concurrent | SVA | on | 0 | 4125  | 21 | 1.7K.. | 3.4K  | 207570 ns | 10292 | off                     | assert( @(posedge clk) disable iff (...) | ✓ |



# WRAPPER\_COVERGROUPS

|                                           |     |           |         |             |   |         |
|-------------------------------------------|-----|-----------|---------|-------------|---|---------|
| /spi_cvg_pkg/spi_cover_gp                 |     |           |         | 100.00%     |   |         |
| TYPE cvr_gp                               | 100 | 100.00... | 100.00% | <div></div> | ✓ | auto(0) |
| CVP cvr_gp::rx_data_all                   | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| CVP cvr_gp::SS_n_seq                      | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| CVP cvr_gp::MOSI_SEQ                      | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| CROSS cvr_gp::{#cross__0#}                | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| + INST \spi_cvg_pkg::spi_cover_gp::cvr_gp | 100 | 100.00... | 100.00% | <div></div> | ✓ | 0       |
| /ram_cvg_pkg/ram_cover_gp                 |     |           |         | 100.00%     |   |         |
| TYPE cvr_gp                               | 100 | 100.00... | 100.00% | <div></div> | ✓ | auto(1) |
| CVP cvr_gp::din_check                     | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| bin wr_add                                | 1   | 100.00... | 11274   | <div></div> | ✓ |         |
| bin wr_data                               | 1   | 100.00... | 5020    | <div></div> | ✓ |         |
| bin read_address                          | 1   | 100.00... | 7523    | <div></div> | ✓ |         |
| bin read_data                             | 1   | 100.00... | 6283    | <div></div> | ✓ |         |
| bin trans_WA_WD                           | 1   | 100.00... | 5020    | <div></div> | ✓ |         |
| bin trans_RA_RD                           | 1   | 100.00... | 5020    | <div></div> | ✓ |         |
| bin trans                                 | 1   | 100.00... | 5020    | <div></div> | ✓ |         |
| CVP cvr_gp::rx_check                      | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| bin rx_valid_0                            | 1   | 100.00... | 25074   | <div></div> | ✓ |         |
| bin rx_valid_1                            | 1   | 100.00... | 5026    | <div></div> | ✓ |         |
| CVP cvr_gp::tx_check                      | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| bin tx_valid_0                            | 1   | 100.00... | 28894   | <div></div> | ✓ |         |
| bin tx_valid_1                            | 1   | 100.00... | 1206    | <div></div> | ✓ |         |
| CROSS cvr_gp::{#cross__0#}                | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| bin <rx_valid_1,trans>                    | 1   | 100.00... | 820     | <div></div> | ✓ |         |
| bin <rx_valid_0,trans>                    | 1   | 100.00... | 4200    | <div></div> | ✓ |         |
| bin <rx_valid_1,trans_RA_RD>              | 1   | 100.00... | 820     | <div></div> | ✓ |         |
| bin <rx_valid_0,trans_RA_RD>              | 1   | 100.00... | 4200    | <div></div> | ✓ |         |
| bin <rx_valid_1,trans_WA_WD>              | 1   | 100.00... | 820     | <div></div> | ✓ |         |
| bin <rx_valid_0,trans_WA_WD>              | 1   | 100.00... | 4200    | <div></div> | ✓ |         |
| bin <rx_valid_1,read_data>                | 1   | 100.00... | 1228    | <div></div> | ✓ |         |
| bin <rx_valid_0,read_data>                | 1   | 100.00... | 5055    | <div></div> | ✓ |         |
| bin <rx_valid_1,read_address>             | 1   | 100.00... | 2199    | <div></div> | ✓ |         |
| bin <rx_valid_0,read_address>             | 1   | 100.00... | 5324    | <div></div> | ✓ |         |
| bin <rx_valid_1,wr_data>                  | 1   | 100.00... | 820     | <div></div> | ✓ |         |
| bin <rx_valid_0,wr_data>                  | 1   | 100.00... | 4200    | <div></div> | ✓ |         |
| bin <rx_valid_1,wr_add>                   | 1   | 100.00... | 779     | <div></div> | ✓ |         |
| bin <rx_valid_0,wr_add>                   | 1   | 100.00... | 10495   | <div></div> | ✓ |         |
| CROSS cvr_gp::{#cross__1#}                | 100 | 100.00... | 100.00% | <div></div> | ✓ |         |
| bin dinXrx                                | 1   | 100.00... | 854     | <div></div> | ✓ |         |

# WRAPPER\_COVERAGE



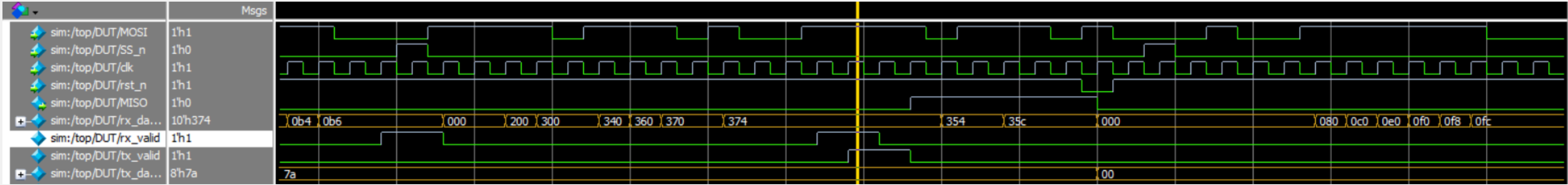
# WRAPPER\_UVM\_MESSAGE

```
# UVM_INFO wrapper_test.sv(64) @ 2000: uvm_test_top [run_phase] rst_end
# UVM_INFO wrapper_test.sv(65) @ 2000: uvm_test_top [run_phase] wr_begin
# UVM_INFO wrapper_test.sv(67) @ 202000: uvm_test_top [run_phase] wr_end
# UVM_INFO wrapper_test.sv(68) @ 202000: uvm_test_top [run_phase] rd_begin
# UVM_INFO wrapper_test.sv(70) @ 402000: uvm_test_top [run_phase] rd_end
# UVM_INFO wrapper_test.sv(71) @ 402000: uvm_test_top [run_phase] wr_rd_begin
# UVM_INFO wrapper_test.sv(73) @ 602000: uvm_test_top [run_phase] wr_rd_end
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 602000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ram_sb.sv(46) @ 602000: uvm_test_top.ram_env_comp.sb [report_phase] Total correct:      30100
# UVM_INFO ram_sb.sv(47) @ 602000: uvm_test_top.ram_env_comp.sb [report_phase] Total false:      0
# UVM_INFO spi_sb.sv(47) @ 602000: uvm_test_top.spi_env_comp.sb [report_phase] Total correct:      30100
# UVM_INFO spi_sb.sv(48) @ 602000: uvm_test_top.spi_env_comp.sb [report_phase] Total false:      0
# UVM_INFO wrapper_sb.sv(49) @ 602000: uvm_test_top.wrapper_env_comp.sb [report_phase] Total correct:      30100
# UVM_INFO wrapper_sb.sv(50) @ 602000: uvm_test_top.wrapper_env_comp.sb [report_phase] Total false:      0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    18
# UVM_WARNING :    2
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]    2
# [RNTST]        1
# [TEST_DONE]    1
# [TPRGED]       2
# [report_phase] 6
# [run_phase]    8
# ** Note: $finish      : C:/questasim64_2021.1/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#   Time: 602 us  Iteration: 61  Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

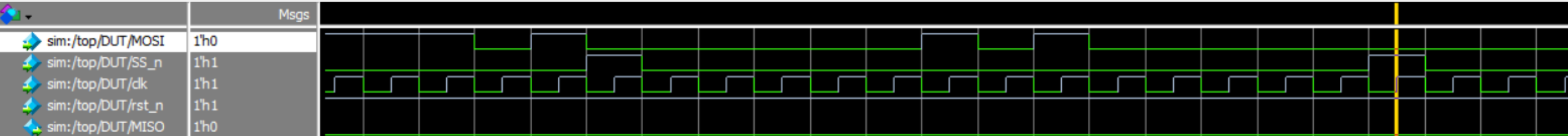


# WRAPPER\_WAVEFORMS

## READ\_DATA\_SEQ

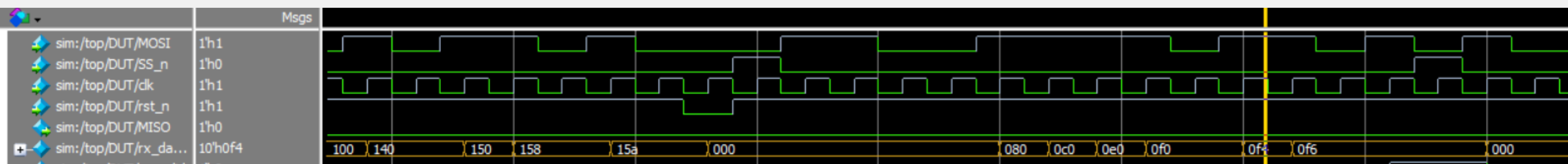


## WR\_ADD\_SEQ



# WRAPPER\_TOP

## RD\_ADD\_SEQ



## WR\_DATA\_SEQ

